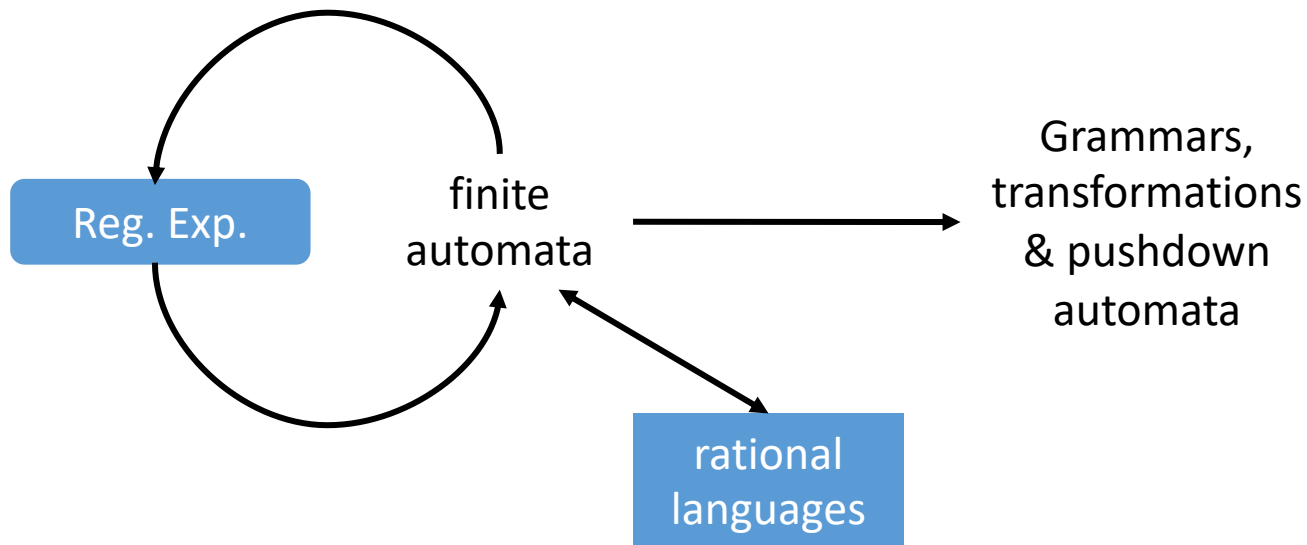
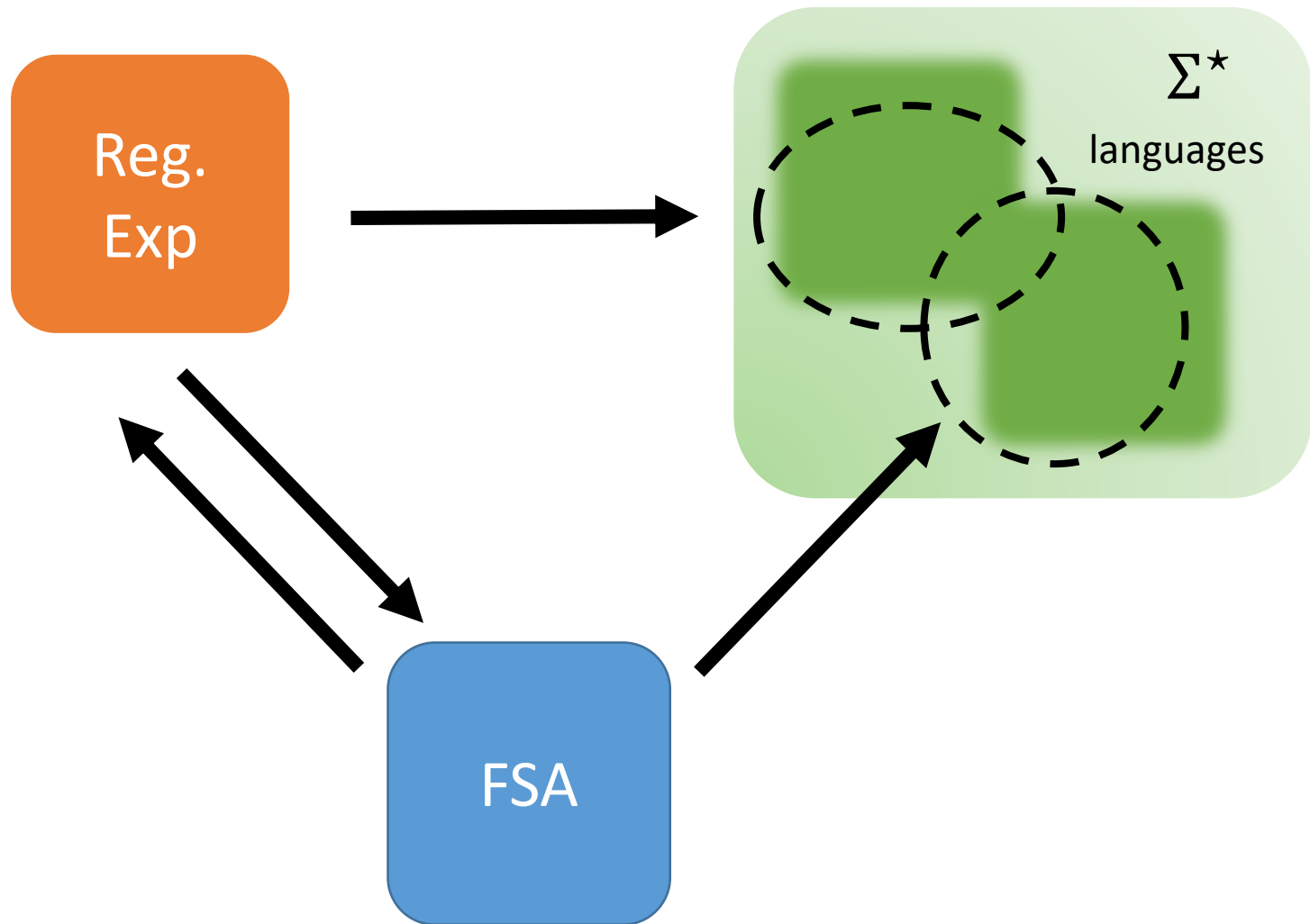


Regular Expressions and Rational Languages



What are we proving today ?



Some people, when confronted with a problem, think "I know, I'll use regular expressions." Now they have two problems.

alt.religion.emacs

Regular Expressions

Regex 101

Regular Expression

A regular expression (regex) is either:

- the constant $\emptyset$ (matching nothing)
- the constant Λ (matching the empty word)
- a symbol a in Σ
- a sequential composition: $R_1 R_2$
- a union: $R_1 + R_2$
- a repetition: R^*

R_1, R_2 two regexes

Regular Expression

It is an expression built from the syntax:

$$\begin{array}{lcl} R, R_1, R_2, \dots & := & \mathbf{0} \\ & & | \mathbf{1} \\ & & | a \\ & & | R_1 \cdot R_2 \\ & & | R_1 + R_2 \\ & & | R_1^* \end{array}$$

Example: $a^*b + b a$

Regular Expression: notations

- we use indifferently $+$ and $|$ for choice
- we simply write $R_1 R_2$ instead of $R_1 . R_2$
- $[abc]$ means $a + b + c$
- $[a-zA-Z]$ for range of symbols
- $[^ab]$ every symbol but a or b
- $[^]$ every symbol in Σ , so $[^]*$ matches ev.
- R^+ stands for $R . R^*$ (at least one R)
- $R^?$ stands for $1 + R$ (zero or one occ.)

“syntactic sugar”

Languages

quick refresher on words and languages

Monoid Σ^*

- a word w is a sequence of symbols in Σ
- we can concatenate two words $w_1 \cdot w_2$
- $|w|$ is the length of w
- ϵ is the empty sequence, $|\epsilon| = 0$
 ϵ is the identity element, $w \cdot \epsilon = \epsilon \cdot w = w$
- $(\Sigma^*, \cdot, \epsilon)$ is the *free monoid* of Σ
- $*$ is often referred to as the Kleene star

Languages over Σ

Languages are subsets of Σ^*

Some examples of languages:

$\emptyset$	also denoted 0
$\{ \epsilon \}$	also denoted Λ or 1
$\{ a \}$	also denoted a

Natural operations between languages

union:	$\mathcal{L}_1 + \mathcal{L}_2$
concatenation:	$\mathcal{L}_1 \cdot \mathcal{L}_2 = \{ u_1 u_2 \mid u_1 \in \mathcal{L}_1, u_2 \in \mathcal{L}_2 \}$
exponentiation:	$\mathcal{L}^n = \{ u_1 \dots u_n \mid u_i \in \mathcal{L} \}$
	$\mathcal{L}^0 = \mathbf{1}$
Kleene star:	$\mathcal{L}^* = \bigcup_{n \geq 0} \mathcal{L}^n$

Regular Expression: semantics

A regex e defines a language $\mathcal{L}(e)$ over Σ^* with the following interpretation:

$$\mathcal{L}(\mathbf{0}) = \emptyset$$

$$\mathcal{L}(\mathbf{1}) = \{\epsilon\} = \Lambda$$

$$\mathcal{L}(a) = \{a\}$$

$$\mathcal{L}(R_1 R_2) = \mathcal{L}(R_1) \cdot \mathcal{L}(R_2)$$

$$\mathcal{L}(R_1 + R_2) = \mathcal{L}(R_1) \cup \mathcal{L}(R_2)$$

$$\mathcal{L}(R^*) = \mathcal{L}(R)^*$$

Languages over Σ : algebraic laws

$$\mathcal{L} + \mathbf{0} = \mathbf{0} + \mathcal{L} = \mathcal{L}$$

$$\mathcal{L} . \mathbf{1} = \mathbf{1} . \mathcal{L} = \mathcal{L}$$

$$\mathcal{L} . \mathbf{0} = \mathbf{0} . \mathcal{L} = \mathbf{0}$$

$$a^{\star} = \mathbf{1} + a . a^{\star} = \{\epsilon\} + \{a\} + \{aa\} + \dots$$

$$(1 + \mathcal{L})^{\star} = \mathcal{L}^{\star}$$

$\emptyset$ also denoted $\mathbf{0}$

$\{\epsilon\}$ also denoted Λ or $\mathbf{1}$

$\{a\}$ also denoted a

Languages over Σ : algebraic laws

$$\mathcal{L} + \mathcal{L} = \mathcal{L}$$

$$(\mathcal{L}_1 + \mathcal{L}_2) \cdot \mathcal{L}_3 = (\mathcal{L}_1 \cdot \mathcal{L}_3) + (\mathcal{L}_2 \cdot \mathcal{L}_3)$$

$$\mathcal{L}^{\star\star} = \mathcal{L}^{\star}$$

...

Languages: other operations

intersection: $\mathcal{L}_1 \cap \mathcal{L}_2$

complement: $\bar{\mathcal{L}} = \Sigma \setminus \mathcal{L}$

mirror image: $\tilde{\mathcal{L}} = \{ \tilde{u} \mid u \in \mathcal{L} \}$

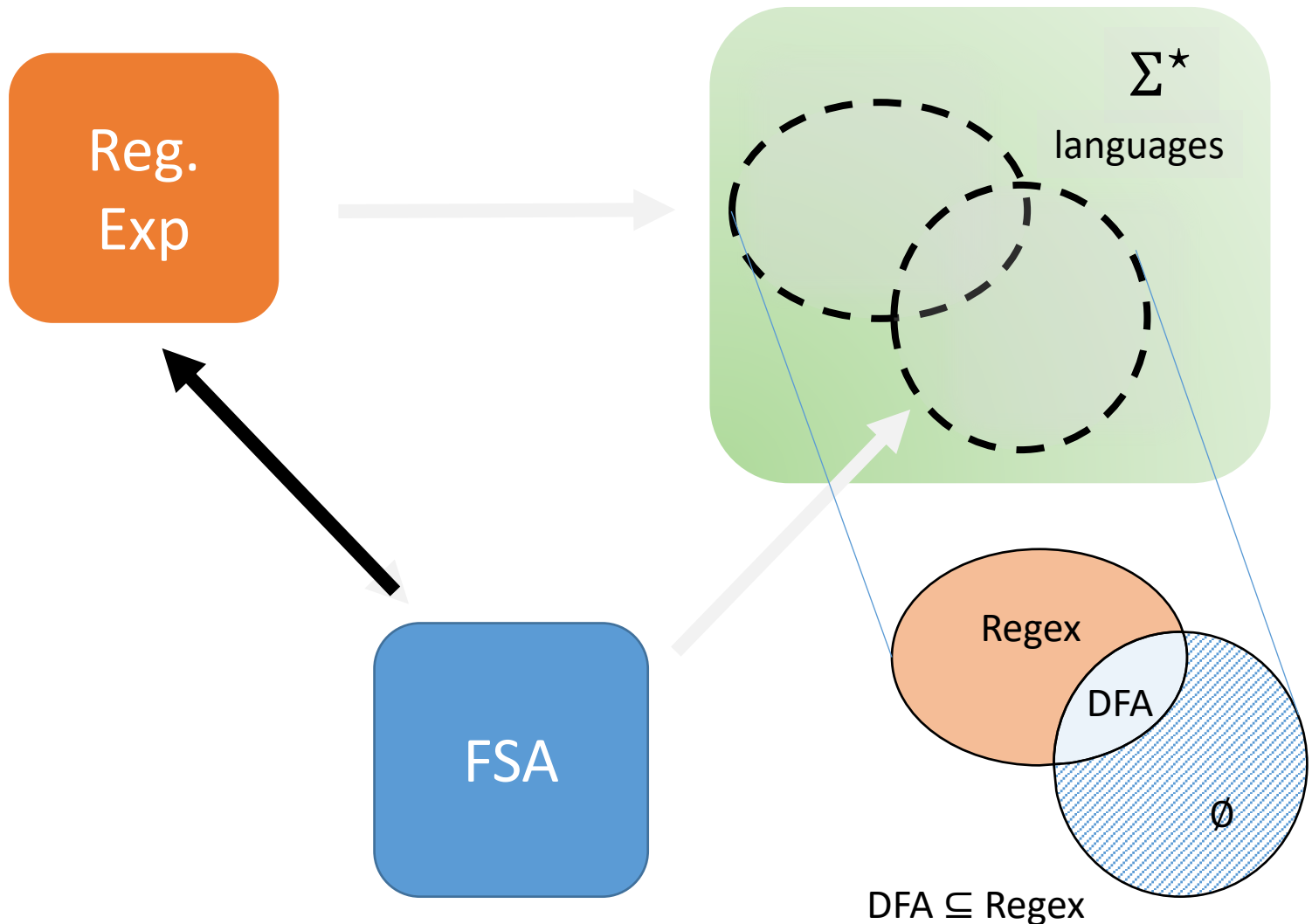
residual: $u^{-1}\mathcal{L} = \{ v \mid u.v \in \mathcal{L} \}$

The notion of residuals is very important to understand the power of DFA.

$(con)^{-1} . \mathcal{L}$

computing
concept
concern
concurrency
concurrent
conference
confuses
connectivity
consistent
consortium
constraint
constructed
context
continues
contradicting
contrast
contribution
control
conventional
cooperates

What are we proving next ?



NFA $\Rightarrow$ Regex

sémantique

Intuition

- Call $\mathcal{A}(i, j)$ the language of words recognized by $\mathcal{A}$ when starting in state i and ending in j
- Call $\mathcal{A}(i)$ the language of words recognized by $\mathcal{A}$ if i was the initial state

$$\mathcal{A}(i) = \bigcup_{q_f \in F} \mathcal{A}(i, q_f)$$

- Then: $\mathcal{L}(\mathcal{A}) = \mathcal{A}(q_i)$

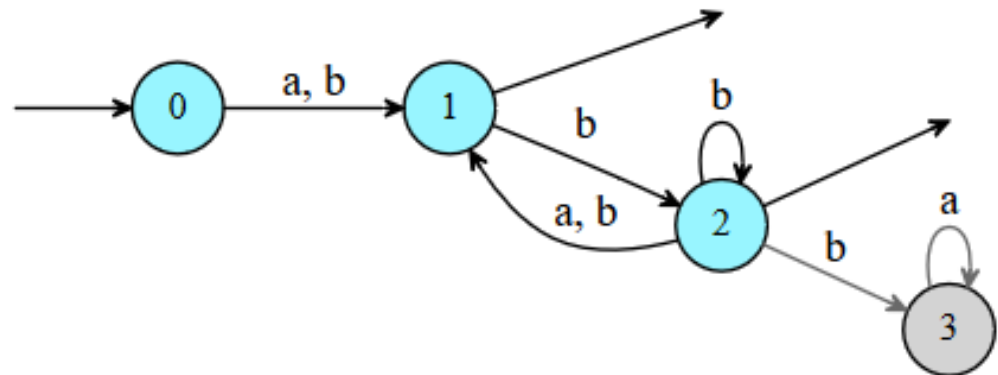
Intuition: matrix format

	\$	0	1	2	3
\$	ϵ	$\top$	-	-	-
0	-	ϵ	a, b	-	-
1	$\top$	-	ϵ	b	-
2	$\top$	-	a, b	ϵ, b	b
3	-	-	-	-	ϵ, a

$\times$ is concatenation

$+$ is set union ($\cup$)

$\star$ is transitive closure



Intuition: matrix format

$\mathcal{A}[i, j] =$ words “accepted” starting from state i , ending in j , in one step ($|w| = 1$)

	\$	0	1	2	3
\$	ϵ	T	-	-	-
0	-	ϵ	a, b	-	-
1	T	-	ϵ	b	-
2	T	-	a, b	ϵ, b	b
3	-	-	-	-	ϵ, a

Intuition: matrix format

$(\mathcal{A} + \mathcal{A}^2) =$ value at $[i, j]$ are the words “accepted” starting from state i , ending in j , in one or two steps ($1 \leq |w| \leq 2$)

	\$	0	1	2	3
\$	ϵ	T	-	-	-
0	-	ϵ	a, b	-	-
1	T	-	ϵ	b	-
2	T	-	a, b	ϵ, b	b
3	-	-	-	-	ϵ, a

Intuition: “linear algebra”

solution to $X = \mathbf{1}_F + \mathcal{A}.X \Rightarrow$ the X_i 's are the words “accepted” starting from state i .

the result in X_i is the set $\mathcal{A}(i)$ defined before.

we have ϵ in X_i iff state i is final.

	\$	0	1	2	3
\$	ϵ	T	-	-	-
0	-	ϵ	a, b	-	-
1	T	-	ϵ	b	-
2	T	-	a, b	ϵ, b	b
3	-	-	-	-	ϵ, a

Arden's rule

Theorem: the language $A^*.B$ is the smallest language that is a solution for X in the (linear) equation:

$$X = A.X + B$$

The solution is unique as soon as $\epsilon \notin A$

This can be extended to sets of equations of the form

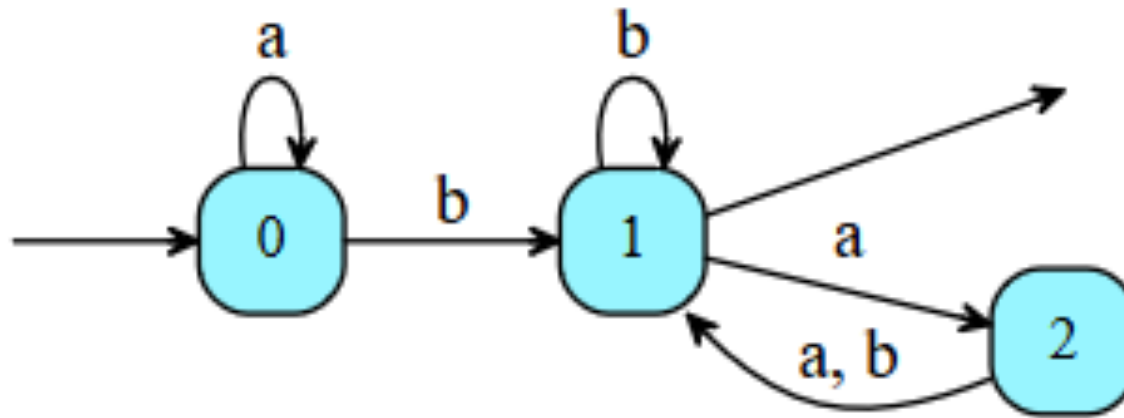
$$X_i = A_{i,1}.X_1 + \dots + A_{i,n}.X_n + B$$

From NFA to Regex

- Use one variable X_i for every state $i \in Q$
- For every transition $\delta(i, a) = j$ add the constraint
$$X_i = a.X_j + X_i$$
- For every state $i \in F$ add the constraint
$$X_i = X_i + \Lambda$$
- Solve for X

we can make use of simplifications in order to have one occurrence of each X_i as a *lhs*, e.g.
 $X = X + X$, or $R.X + R'.X = (R + R').X$, etc.

From NFA to Regex



$$X_0 = a.X_0 + b.X_1$$

$$X_1 = b.X_1 + a X_2 + \Lambda$$

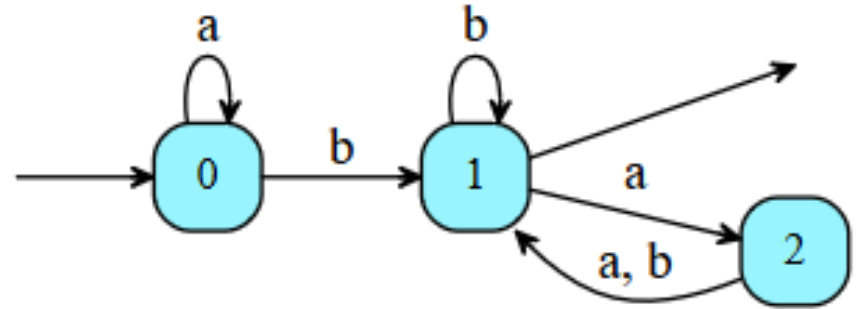
$$X_2 = (a + b).X_1$$

From NFA to Regex

$$X_0 = a.X_0 + b.X_1$$

$$X_1 = b.X_1 + a.X_2 + \Lambda$$

$$X_2 = (a + b).X_1$$

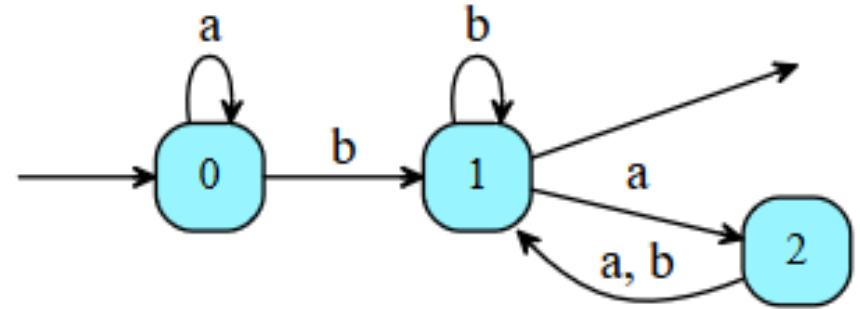


From NFA to Regex

$$X_0 = a.X_0 + b.X_1$$

$$X_1 = b.X_1 + a.X_2 + \Lambda$$

$$X_2 = (a + b).X_1$$



⇓ substitution + factorization

$$X_0 = a.X_0 + b.X_1$$

$$X_1 = b.X_1 + a(a + b).X_1 + \Lambda = R.X_1 + \Lambda$$

$$X_2 = (a + b).X_1$$

where $R = b + a(a + b)$

From NFA to Regex

$$X_0 = a.X_0 + b.X_1$$

$$X_1 = R.X_1 + \Lambda$$

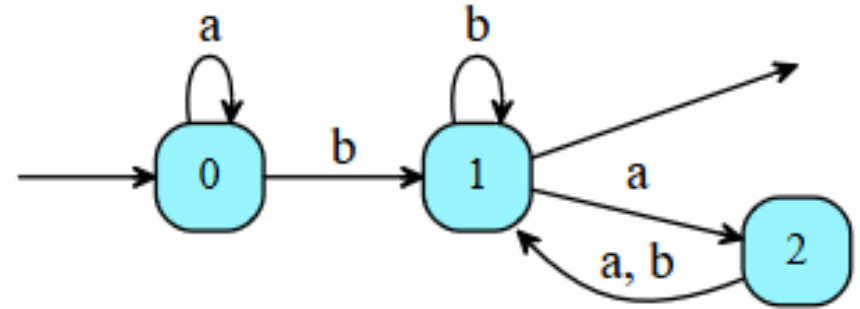
$$X_2 = (a + b).X_1$$

⇓ Arden's rule

$$X_0 = a.X_0 + b.X_1$$

$$X_1 = R^*. \Lambda = R^*$$

$$X_2 = (a + b).X_1$$



where $R = b + a(a + b)$

From NFA to Regex

$$X_0 = a.X_0 + b.X_1$$

$$X_1 = R^*$$

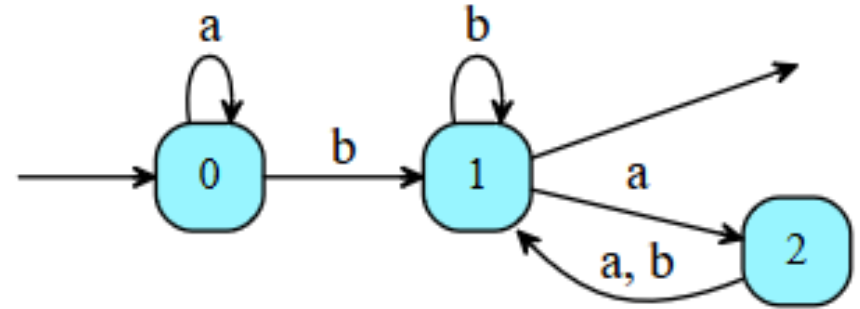
$$X_2 = (a + b).X_1$$

⇓ substitution

$$X_0 = a.X_0 + b.R^*$$

$$X_1 = R^*$$

$$X_2 = (a + b).R^*$$



where $R = b + a(a + b)$

From NFA to Regex

$$X_0 = a.X_0 + b.R^*$$

$$X_1 = R^*$$

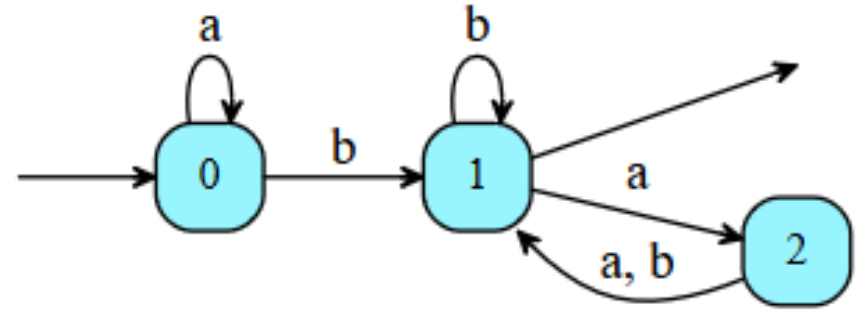
$$X_2 = (a + b).R^*$$

⇓ Arden's rule

$$X_0 = a^*.b.R^*$$

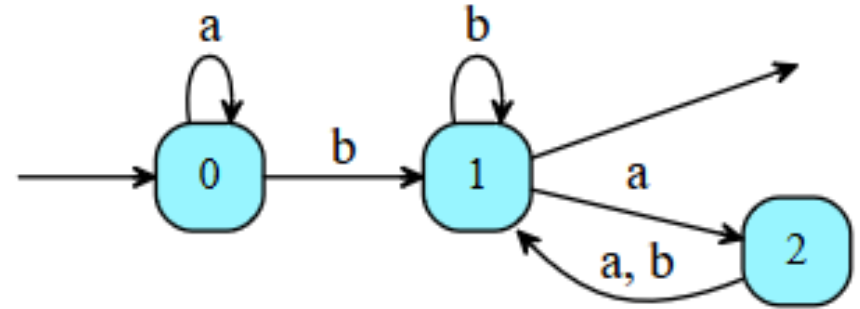
$$X_1 = R^*$$

$$X_2 = (a + b).R^*$$



where $R = b + a(a + b)$

From NFA to Regex



$$X_0 = a^*.b.(b + a(a + b))^*$$

$$X_1 = (b + a(a + b))^*$$

$$X_2 = (a + b).(b + a(a + b))^*$$

where $R = b + a(a + b)$

From NFA to Regex: remarks

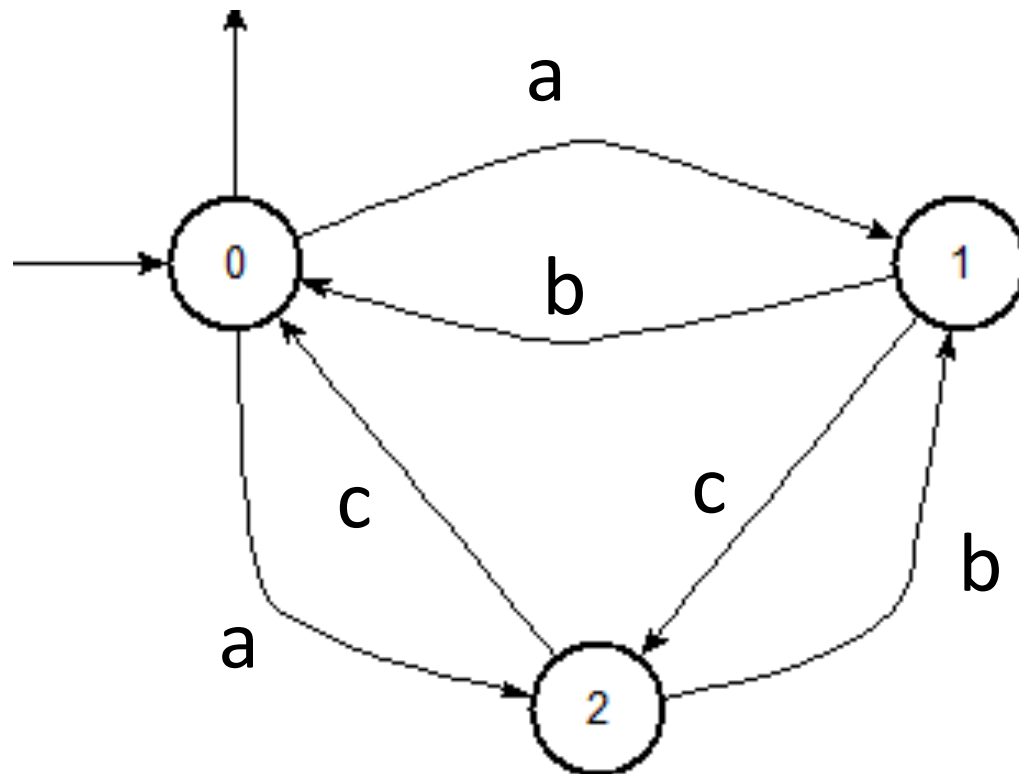
- This construction can be used on automata that are not deterministic and/or not complete
- The size of the resulting regex depends on the order in which we “eliminates” variables
- The resulting regex can be exponentially larger than the size of $\mathcal{A}$

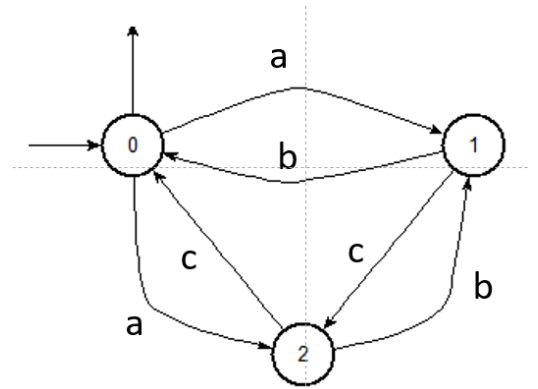
but there are DFA with “small” regex: $(a|b)^*.a.(a|b)^n$

Theorem: we have $\text{NFA} = \text{DFA} \subseteq \text{Regex}$

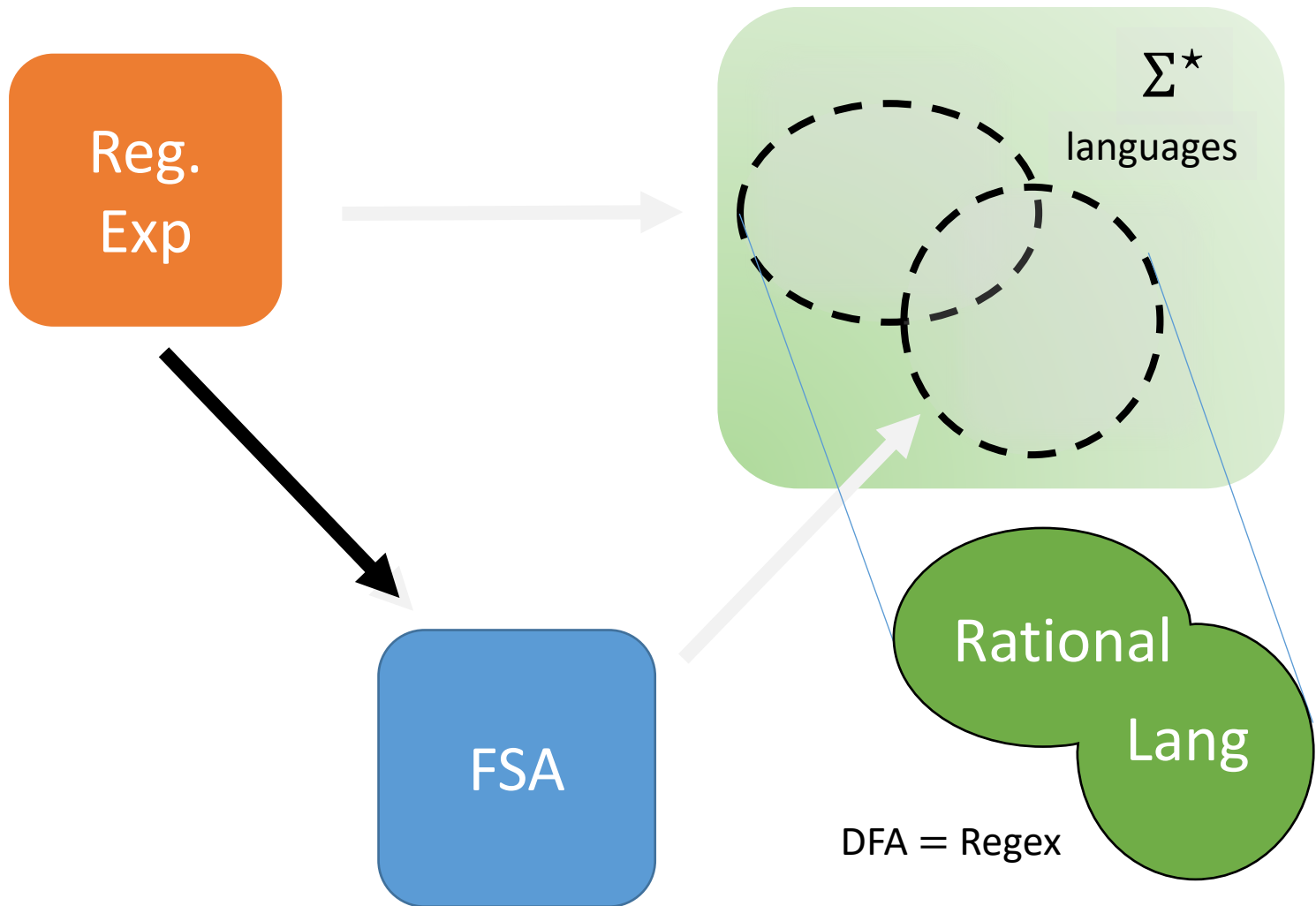
no real applications in practice ?

Example





What are we proving next ?



Automata & Closure properties

Product of automata

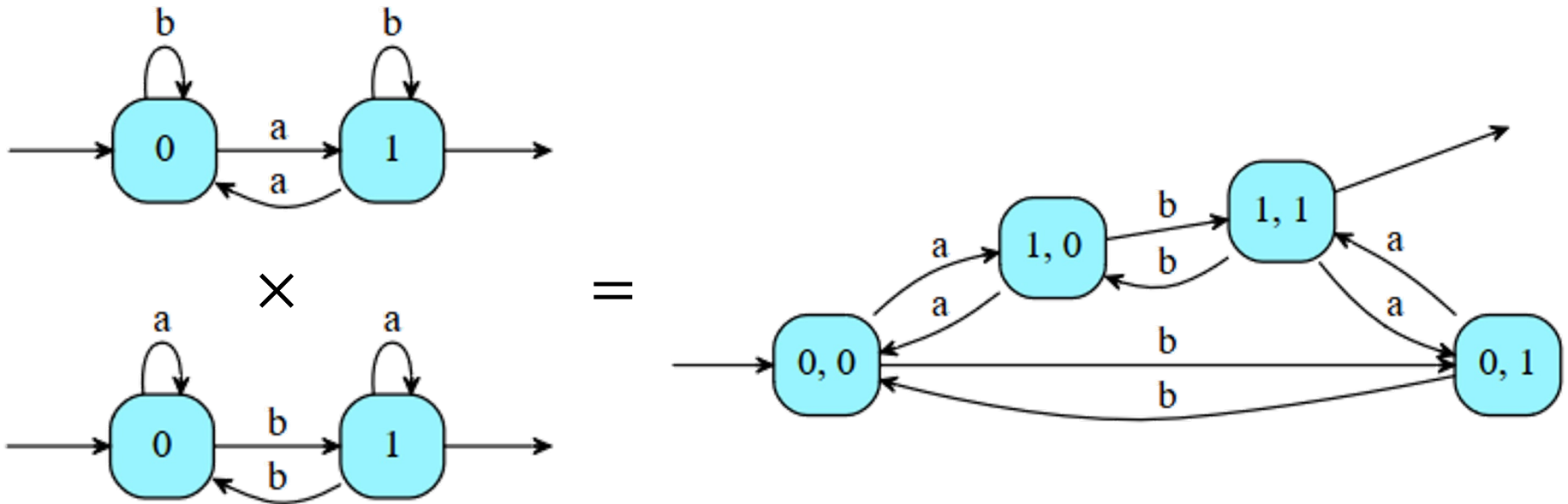
Given two NFA $\mathcal{A}_1$ and $\mathcal{A}_2$, we define the product NFA $\mathcal{A}_1 \times \mathcal{A}_2$ such that

- set of states $Q_1 \times Q_2$
- initial state is (q_1^1, q_1^2)
- final states $F_1 \times F_2$
- $\delta((q_1, q_2), a) = (q'_1, q'_2)$ whenever both $\delta_1(q_1, a) = q'_1$ and $\delta_2(q_2, a) = q'_2$

$$\mathcal{A}_i = (Q_i, \Sigma, \delta_i, q_i^1, F_i), i \in 1..2$$

Product of automata

We can prove that a word u is in $\mathcal{L}(\mathcal{A}_1 \times \mathcal{A}_2)$ iff both $u \in \mathcal{L}(\mathcal{A}_1)$ and $u \in \mathcal{L}(\mathcal{A}_2)$



This gives a simple construction for computing the “intersection” of two languages

The definition still works when $\Sigma_1 \neq \Sigma_2$ ($\Rightarrow$ synchronous product)

Product of automata

Theorem: DFA are closed by intersection.

Therefore Regex are also closed by intersection.

not obvious by direct means !

Complement of an automata

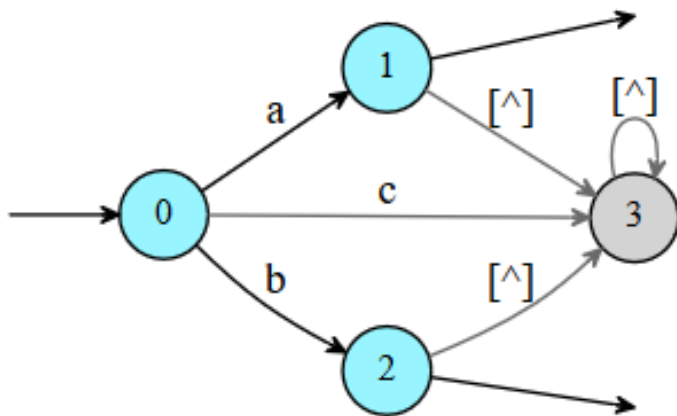
Given a complete + deterministic FSA $\mathcal{A}$, we can define its complement $\mathcal{A}^c$ such that

- set of states Q
- initial states is q_I
- final states $Q \setminus F$
- same δ

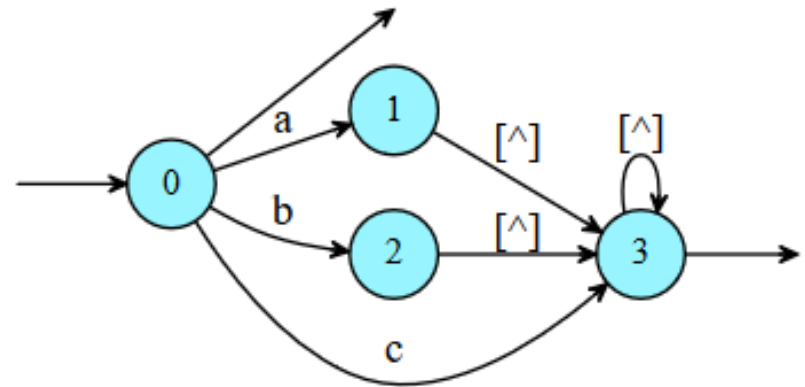
$$\mathcal{A} = (Q, \Sigma, \delta, q_I, F)$$

Complement of an automata

We can prove that a word u is in $\mathcal{L}(\mathcal{A})$ iff $u \notin \mathcal{L}(\mathcal{A}^c)$



$$\mathcal{A} \equiv a + b$$



$$\mathcal{A}^c \equiv \epsilon + c + [\wedge].[\wedge]^*$$

This construction entails $\mathcal{A}$ complete (easy) AND deterministic (costly)

The result is also complete and deterministic

Complement of an automata

Theorem: DFA are closed by complement.

Therefore Regex are also closed by negation.

even less obvious by direct means !

Union of automata

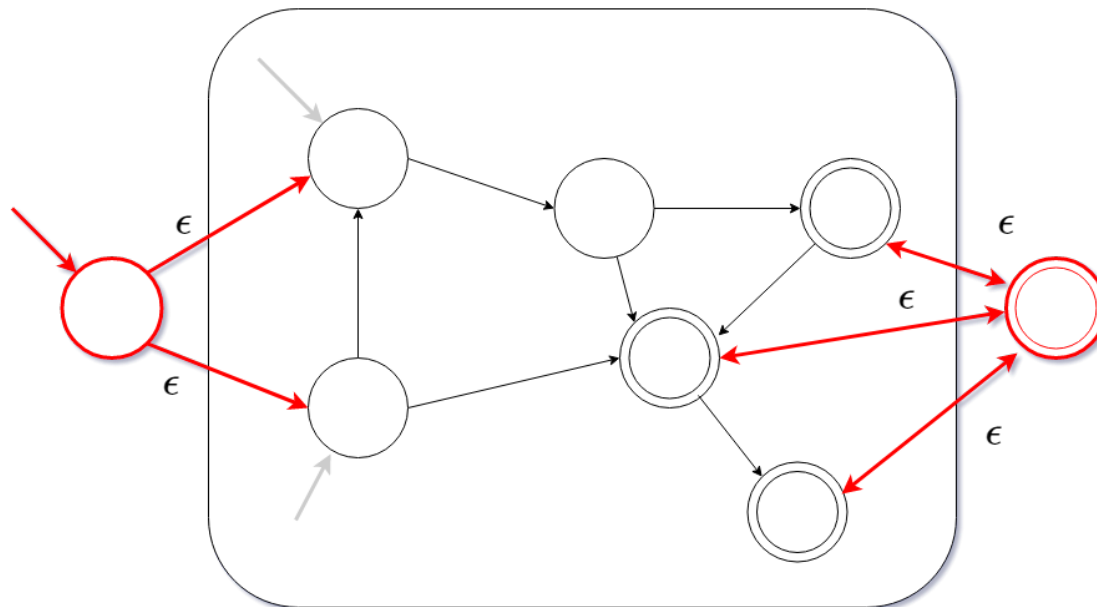
Given two NFA $\mathcal{A}_1$ and $\mathcal{A}_2$, we define the union NFA $\mathcal{A}_1 \cup \mathcal{A}_2$ such that

- set of states $Q_1 \cup Q_2 \cup \{ q_I \}$
- initial state is q_I
- final states $F_1 \cup F_2$
- $\delta(q, a) = q'$ if $\delta_i(q, a) = q'$ for some $i \in 1..2$
- $\delta(q_I, \epsilon) = q_I^1$ and $\delta(q_I, \epsilon) = q_I^2$

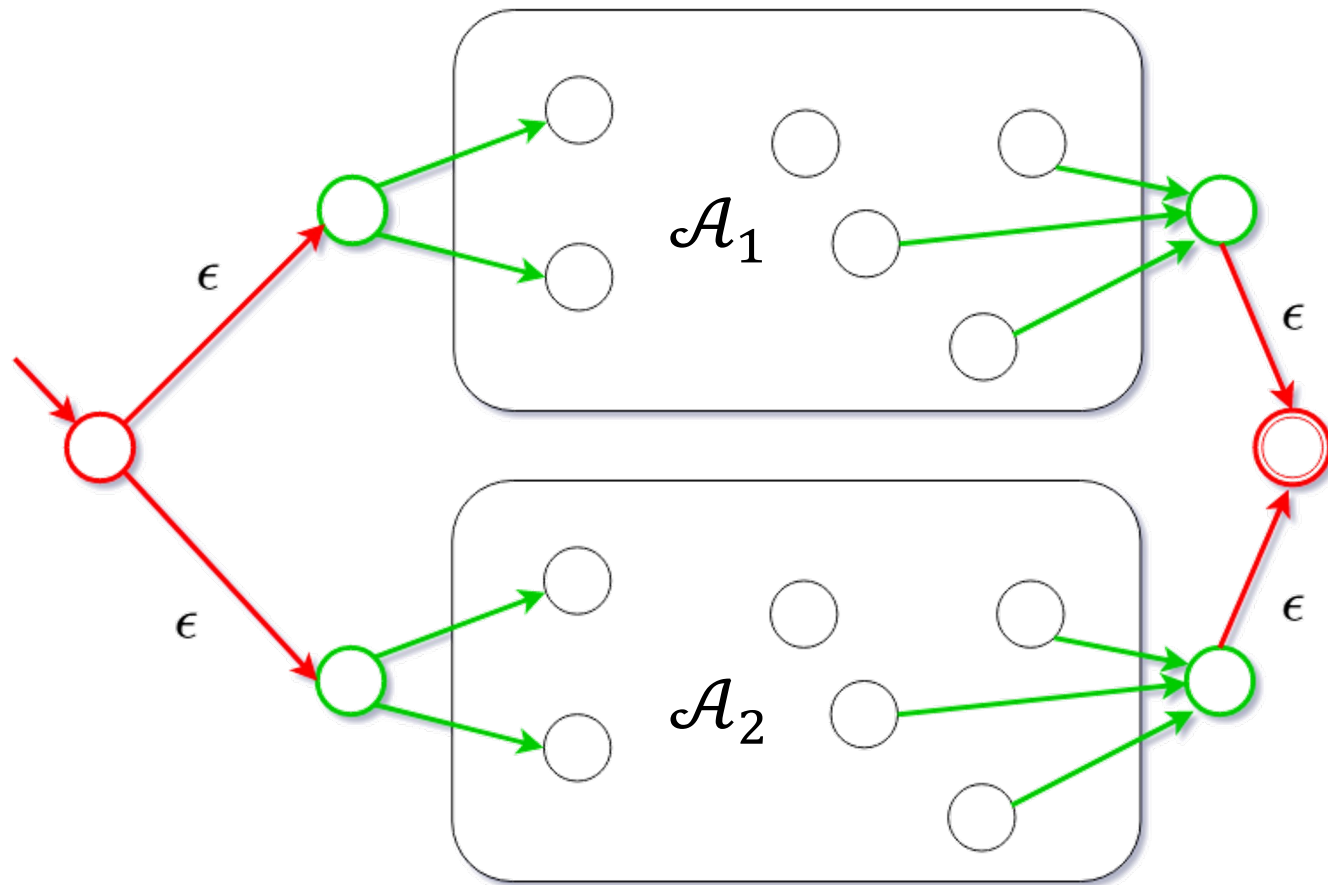
$$\mathcal{A}_i = (Q_i, \Sigma, \delta_i, q_I^i, F_i), i \in 1..2$$

Standard form

- We can always assume a unique initial state
- no incoming transition on the initial state
- a single final state and no back transitions from it



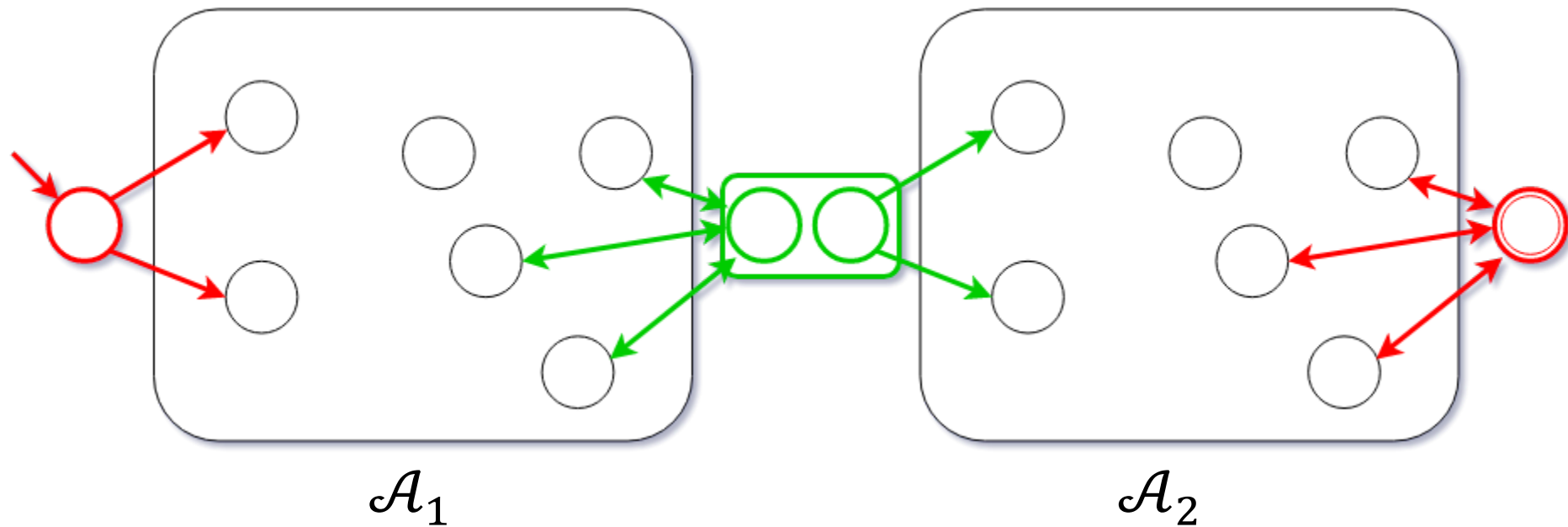
Union of automata



Union of automata

Theorem: DFA are closed by $+$.

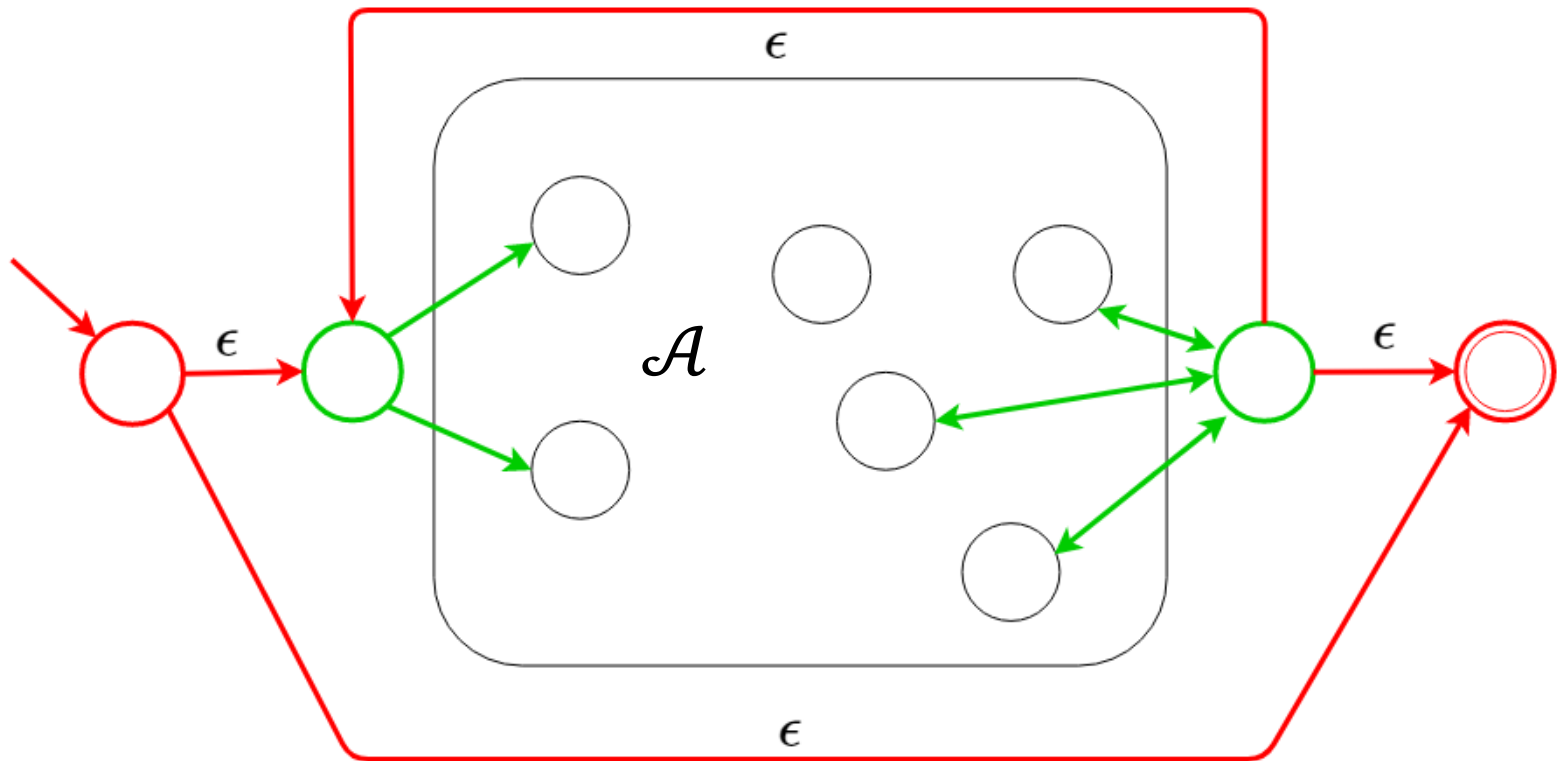
Concatenation of automata



Concatenation of automata

Theorem: DFA are closed by concatenation.

Iteration of automata



Iteration of automata

Theorem: DFA are closed by $\star$.

Closure properties

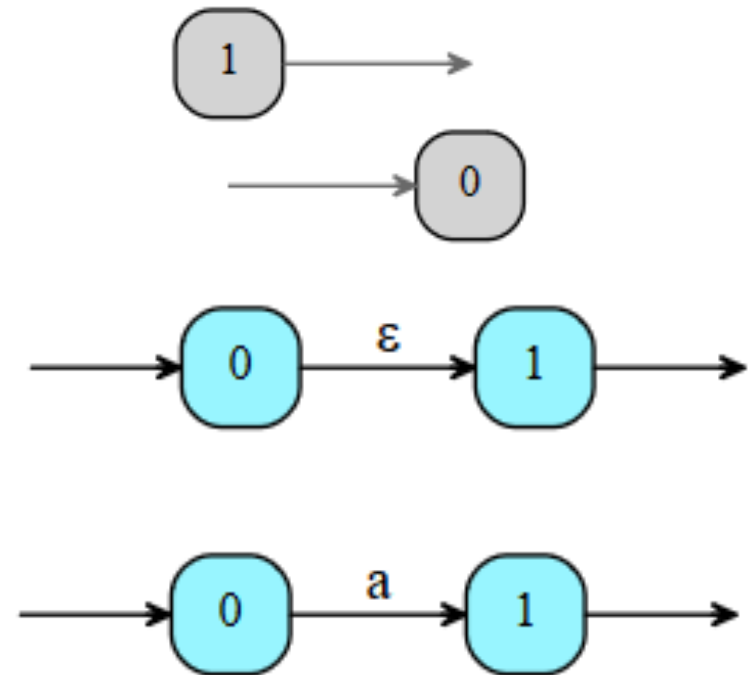
$\Rightarrow$ we prove that
 $\text{Regex} \subseteq \text{DFA}$

DFA are closed by $.$, $+$, and $*$

There is a DFA for **0**

There is a DFA for **1**

There is a DFA for a



Mirror image of automata

Given a complete DFA $\mathcal{A}$, we can define its mirror image $\tilde{\mathcal{A}}$ such that

- set of states Q
- initial states is ...
- final states ...
- transition function is $\tilde{\delta}$ such that ...

left has an exercise !

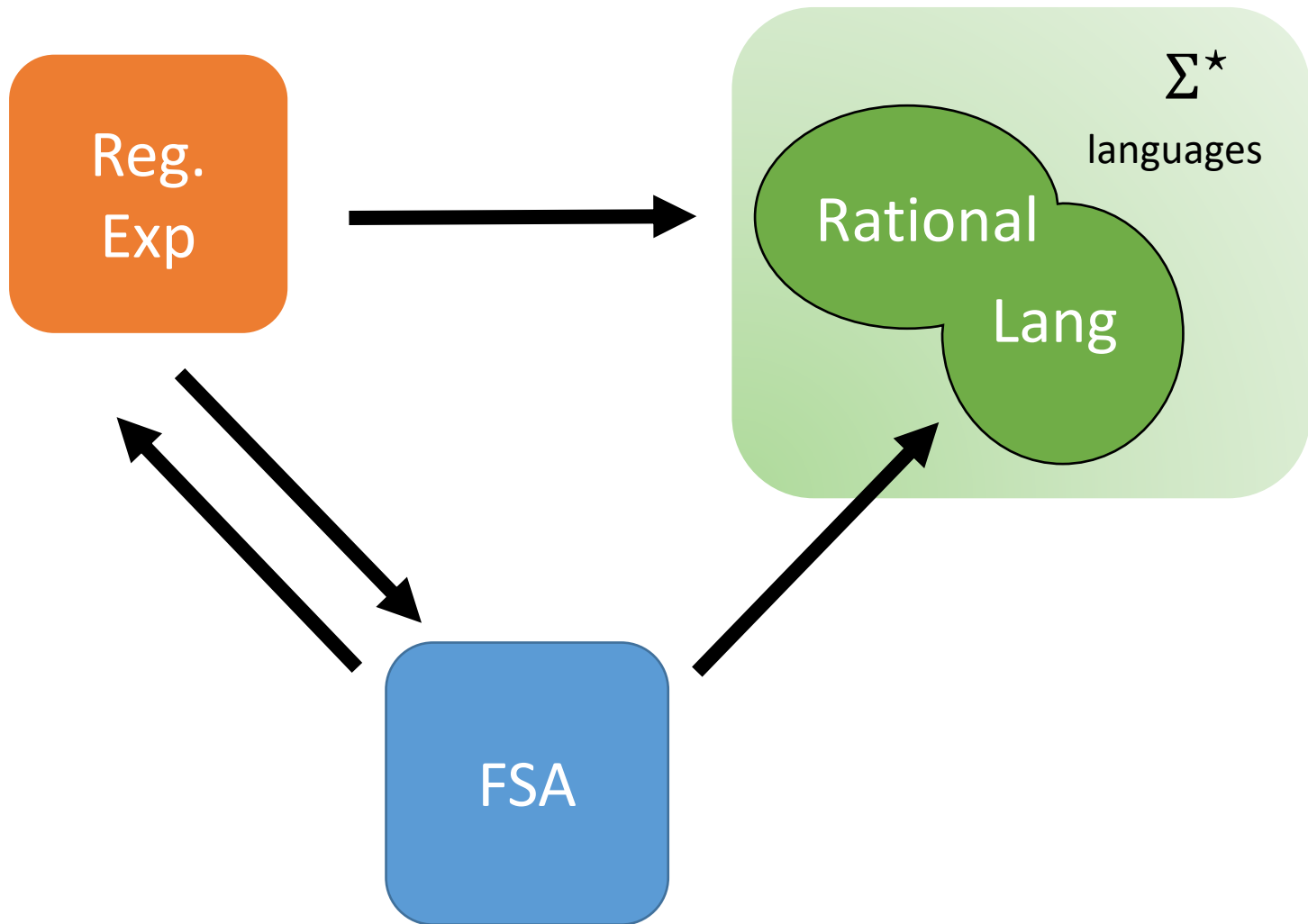
$$\mathcal{A} = (Q, \Sigma, \delta, q_I, F)$$

Shuffle of two automata

- The shuffle of two words, $u \# v$, is the set of words obtained by interlacing u and v
 $\approx$ concatenation for concurrent activities
- The shuffle of two languages $\mathcal{L}_1 \# \mathcal{L}_2$ is the set of words $u_1 \# u_2$ with $u_i \in \mathcal{L}_i, i \in 1..2$
- “DFA” are closed by shuffle

left has an exercise !

What we have proved so far



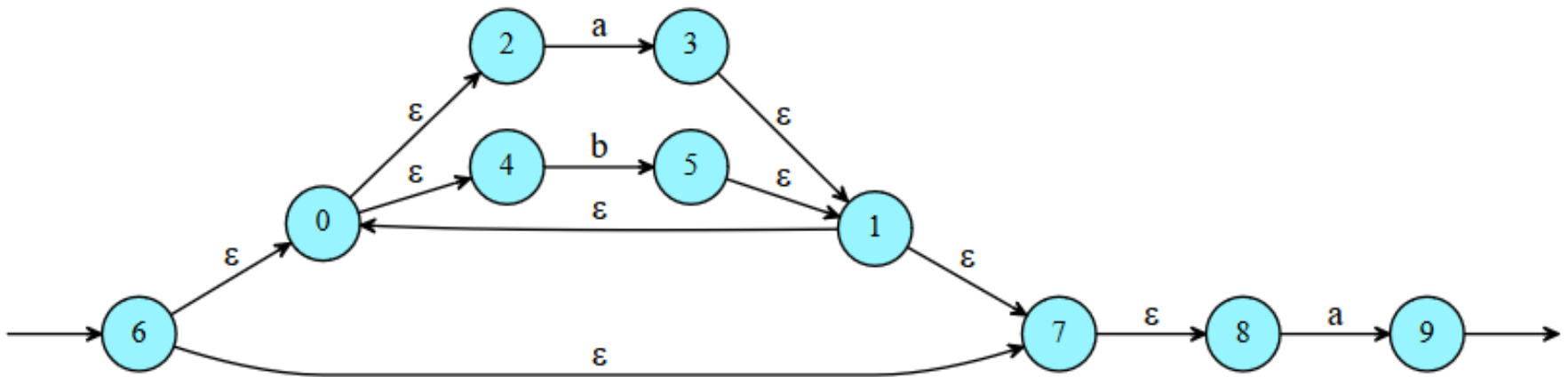
Regex $\Rightarrow$ NFA

Regex $\Rightarrow$ NFA: compositionally

- The previous results give a simple and *compositional* method for computing a NFA equivalent to a regex $\Rightarrow$ Thompson's method
- The result is in standard form
- If regex e has c concatenation and s symbols then the resulting NFA has $2s - c$ states
- Hence we can do pattern matching of e on a word u with “*linear*” complexity $O(|e|^2 \cdot |u|)$

Regex $\Rightarrow$ NFA: compositionally

- The resulting NFA has a lot of ϵ -transitions $\Rightarrow$ we can do better (see Glushkov's construction)
- The result is non-det. $\Rightarrow$ we can do better



Thompson's construction for $(a + b)^*.a$

Example

- Try to match word $a a b$ with the regex $a^* . a^* . a . b$ using Thompson's method

It is possible to do a little better by compressing the NFA first, see: Chia-Hsiang, Chang, and Robert Paige. "From regular expressions to DFA's using compressed NFA's." Theoretical Computer Science 178.1-2 (1997): 1-36.

See also: <https://swtch.com/~rsc/regexp/regexp1.html>

Regex $\Rightarrow$ Automata

La Minute nécessaire de monsieur Cyclopède

Compiling Regex

- Even this simple construction is useful
 - Thompson used it to program grep
- Many useful regex

Phone #: `^\>(*\d{3}\)( |-)*\d{3}( |-)*\d{4}$`

basename: `[^\\]+\\*$`

- Modern languages extend the power of regex
 - example: *capture groups* and *backreferences*,
`\b (\w+) \b \s+ \1 \b` captures repeated word

see <https://blog.codinghorror.com/regex-use-vs-regex-abuse/>

Abusing Regex

A 6343 of matching addresses

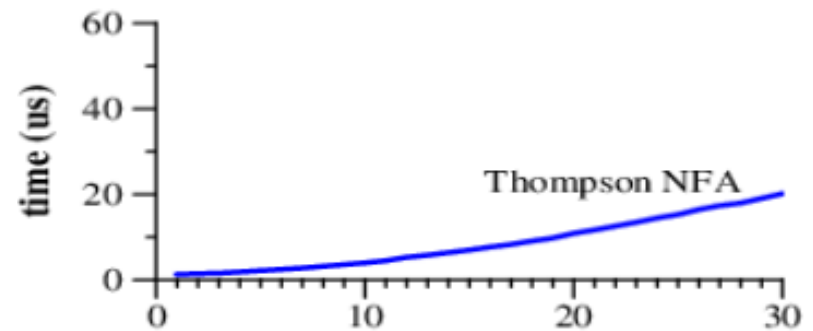
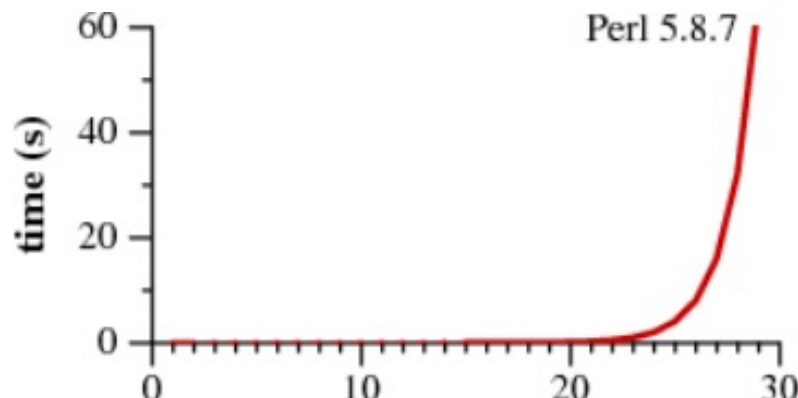
The author "[this] so limits of work do with r

A 6343 characters long regex matching valid RFC822 email addresses

The author comments that:
“[this] somewhat pushes the limits of what it is sensible to do with regular expressions”

Compiling Regex

- Modern languages extend the power of regex
example: *capture groups* and *backreferences*,
`\b (\w+) \b \s+ \1 \b`
- ... which means many “regex API” do not use plain automata



time to match pattern $(a^?)^n a^n$ against the word a^n

Regex $\Rightarrow$ DFA

Brzozowski derivative

Residuals and derivative

Residual: $u^{-1}\mathcal{L} = \{ v \mid u.v \in \mathcal{L} \}$

we have $(a.u)^{-1}\mathcal{L} = u^{-1}(a^{-1}\mathcal{L})$

the language $a^{-1}\mathcal{L}$ is called a *derivative*

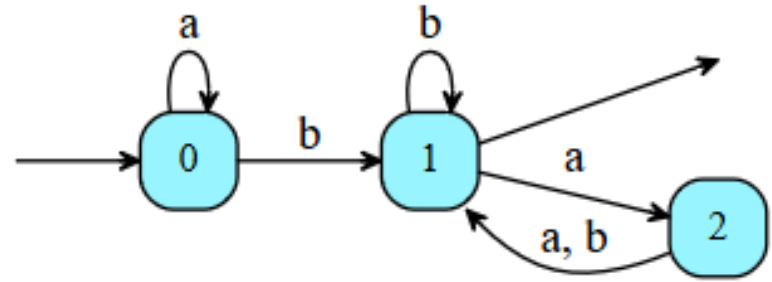
computing
concept
concern
concurrency
concurrent
conference
confuses
connectivity
consistent
consortium
constraint
constructed
context
continues
contradicting
contrast
contribution
control
conventional
cooperates

think $(f \circ g)^{-1} = g^{-1} \circ f^{-1}$

$(con)^{-1} . \mathcal{L}$

Functions in $\Sigma^* \rightarrow \Sigma^*$

Intuition



- Take a complete DFA $\mathcal{A}$ with n states
- Each word u leads to one state, say q_u , meaning that $(q_I, u) \Rightarrow q_u$
- The set of words accepted by $\mathcal{A}$ when starting from q_u , denoted $\mathcal{A}(q_u)$ before, is exactly $u^{-1}\mathcal{L}(\mathcal{A})$

There is a mapping (surjection) from Q
 $\mapsto$ the set of residuals of $\mathcal{A}$

Intuition (2)

- The set of words accepted by $\mathcal{A}$ when starting from q_u , denoted $\mathcal{A}(q_u)$ before, is exactly $u^{-1}\mathcal{L}(\mathcal{A})$
- If $\delta(q, a) = q'$ then: $\mathcal{A}(q) = a.\mathcal{A}(q')$ and
 $\mathcal{A}(q') = a^{-1}\mathcal{A}(q)$

There is a mapping (bijection) between the transitions in $\mathcal{A} \mapsto$ and the set of residuals of $\mathcal{A}$

the relation is $\propto$ in the case of a NFA

Derivatives of a Regex

- We can define the notion of residuals/derivates directly at the level of Regex
- $D_a(e)$ = regex matching the words in $a^{-1}\mathcal{L}(e)$
- $D_a(e)$ = “*what we match in e after reading an a* ”

Derivatives of a Regex

- $D_a(\mathbf{0}) = \mathbf{0}$
- $D_a(\mathbf{1}) = \mathbf{0}$
- $D_a(a) = \mathbf{1}$
- $D_a(b) = \mathbf{0}$
- $D_a(e_1 + e_2) = D_a(e_1) + D_a(e_2)$
- $D_a(e_1 \cdot e_2) = D_a(e_1) \cdot e_2 + \epsilon^?(e_1) \cdot D_a(e_2)$
- $D_a(e^*) = D_a(e) \cdot e^*$

where $\epsilon^?(e) = \mathbf{1}$ if $\epsilon \in e$, otherwise $\mathbf{0}$

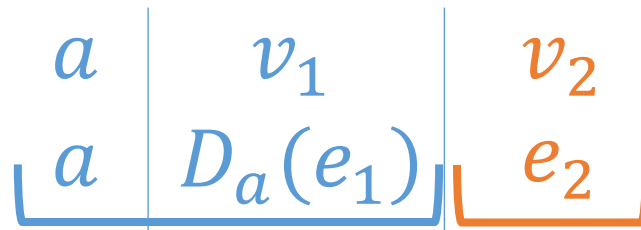
Derivatives $\approx$ Differentials

- f constant $\frac{dc}{dx} = \mathbf{0}$
- f linear $\frac{df}{dx} = \mathbf{1}$ or $\mathbf{0}$ (think of $f(x, y) = y$)
- addition: $\frac{d(f+g)}{dx} = \frac{df}{dx} + \frac{dg}{dx}$
- product: $\frac{d(f \cdot g)}{dx} = \frac{df}{dx} \cdot g + f \cdot \frac{dg}{dx}$
- iteration: $f^* = \mathbf{1} + f \cdot f^*$

Derivatives: rule for $\cdot$

- What are the words, matched by $e_1 \cdot e_2$, starting with an a (this is $D_a(e_1 \cdot e_2)$).
- These are words of the form $u = a v_1 v_2$

(1) e_1 matches a (and the start of u)



(2) e_1 can match ϵ , then we can also match u with e_2



Derivatives: examples

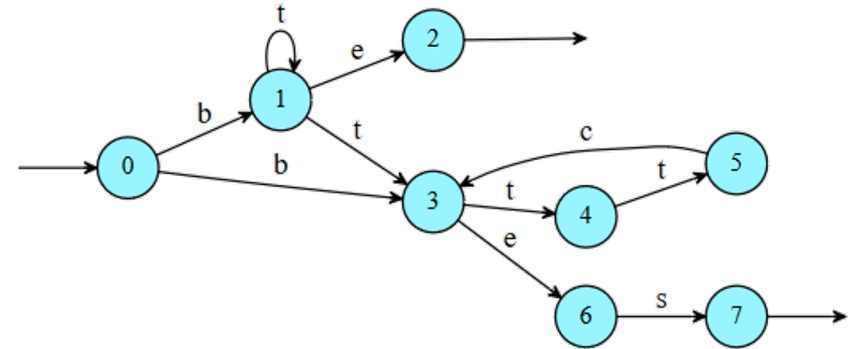
- $D_a((a + b)^n) = (a + b)^{n-1}$ with $n \geq 1$
- $D_a((a + b)^*) = (a + b)^*$
- De Bruijn expression: $J_n = (a + b)^* a (a + b)^n$
- $D_a(J_n) = J_n + \mathbf{1} . D_a(a(a + b)^n)$
 $= J_n + (a + b)^n$
- $D_b(J_n) = J_n + \mathbf{1} . D_b(a(a + b)^n)$
 $= J_n + \mathbf{0}$

Regex $\Rightarrow$ DFA: Brzozowski

- We can build an DFA from e_0 using derivatives
- We have an initial state $\equiv e_0$
- We have one state for each residuals of e_0
 - there are finitely many (TBD)
- We have a transition from e_i to e_j , labeled a , whenever $D_a(e_i) = e_j$
 - ensure both deterministic + complete
- Final states are regex matching ϵ (i.e. $\epsilon^?(e_i) = \mathbf{1}$)
 - because we can stop there !

drawback: two derivatives may correspond to = languages

Example



communication protocol:

$$b t^* e + (b + b t^+)(t t c)^* e s$$

Rational Languages

Language: $\equiv$ modulo $\mathcal{L}$

residual: $u^{-1}\mathcal{L} = \{ v \mid u.v \in \mathcal{L} \}$

Definition: We say that $u \equiv_L v$ if u and v have the same residuals in $\mathcal{L}$.

$$u \equiv_L v \quad \text{iff} \quad u^{-1}\mathcal{L} = v^{-1}\mathcal{L}$$

Rational Languages

Definition: a language $\mathcal{L}$ is said to be *rational*, $\mathcal{L} \in \text{Rat}$, if $\exists \mathcal{A}$ such that $\mathcal{L} = \mathcal{L}(\mathcal{A})$

or equivalently

Definition: a language $\mathcal{L}$ is said to be *rational*, $\mathcal{L} \in \text{Rat}$, if $\exists e$ such that $\mathcal{L} = \mathcal{L}(e)$

Rational Languages

Definition: a language $\mathcal{L}$ is said to be *rational*, $\mathcal{L} \in \text{Rat}$, if $\exists \mathcal{A}$ such that $\mathcal{L} = \mathcal{L}(\mathcal{A})$

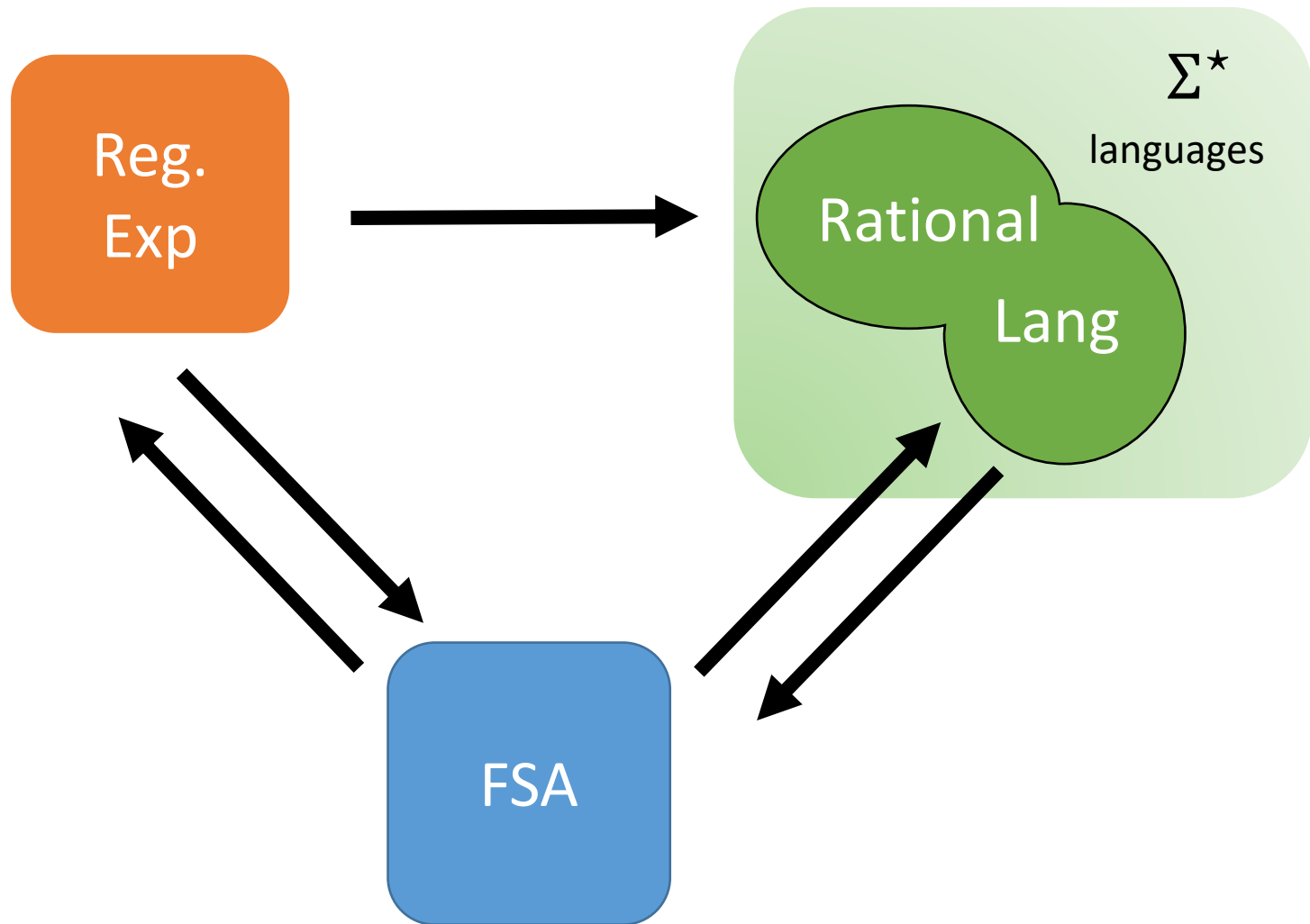
Theorem (Myhill-Nerode): a language $\mathcal{L}$ is *rational* iff the relation $\equiv_L$ has a finite number of equivalent classes.

moreover each class $\mapsto$ (bijectively) the states of the *minimal DFA* accepting $\mathcal{L}$.

Why is it called rational

$$1/7 = 0 . \overbrace{142857}^p 142857 142...$$

What we have proved



Some languages are not rational

Classical example: take $S_{n,n} = \{ a^n b^n \mid n \geq 0 \}$, the language of strings in $a^* b^*$ with as many a 's than b 's

$$S_{n,n} = \{ \epsilon, ab, aabb, \dots \}$$

$$D_a(S_{n,n}) = S_{n,n+1} = \{ \epsilon, b, abb, aabbb, \dots \}$$

$$D_{aa}(S_{n,n}) = S_{n,n+2} = \{ \epsilon, bb, abbb, \dots \}$$

...

$$D_{a^j}(S_{n,n}) = S_{n,n+j} = \{ \epsilon, b^j, ab^{j+1}, \dots \}$$

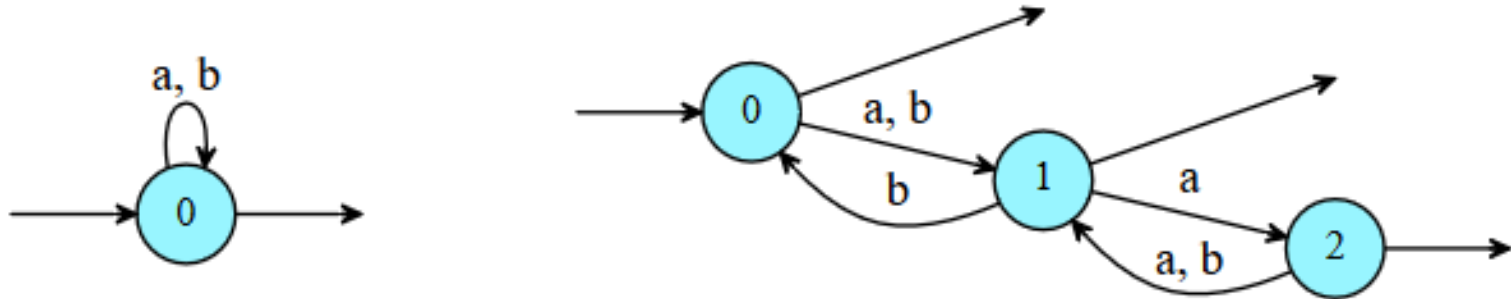
all those derivatives are $\neq$

Automata

Equivalence and minimization

Minimality and DFA

- There can be many equivalent DFA



- A sureway method for minimality is to use $\equiv_L$
- We can also use *partition refining* methods to find equivalent states in a DFA

Minimization $\equiv$ Equivalence

- Minimal DFA $\Rightarrow$ we have a *canonical DFA* for each rational language (up-to naming of the states)
- Hence, to test if two NFA $\mathcal{A}$ and $\mathcal{B}$ are equivalent (they accept the same language), we can just test:

$$\min(\det(\mathcal{A})) \stackrel{?}{=} \min(\det(\mathcal{B}))$$

- also work with regex !

Minimizing a DFA

- We want to find equivalent states, $\equiv$
 $p \equiv q$ meaning $\mathcal{A}(p) = \mathcal{A}(q)$
- We have two necessary conditions
 1. If $p \equiv q$ and p a final state ($\epsilon \in \mathcal{A}(p)$) then q final
 2. if $p \equiv q$ then $\delta(p, a) \equiv \delta(q, a)$ (same residuals)

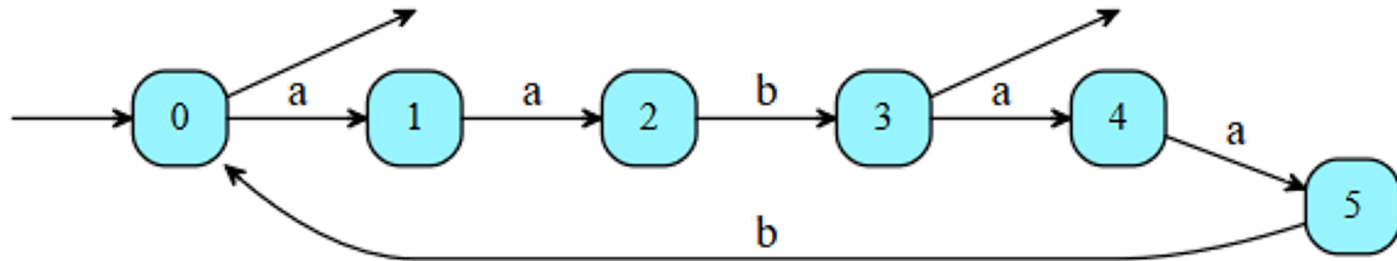
Minimizing a DFA: Idea

We try to compute the equivalence classes for $\equiv$

1. start: assume all states are equivalent
2. repeat: split a class when you find two states with different transitions (up-to $\equiv$)
3. end: until we reach a fix point

$\Rightarrow$ all the states in the same partition are
non distinguishable

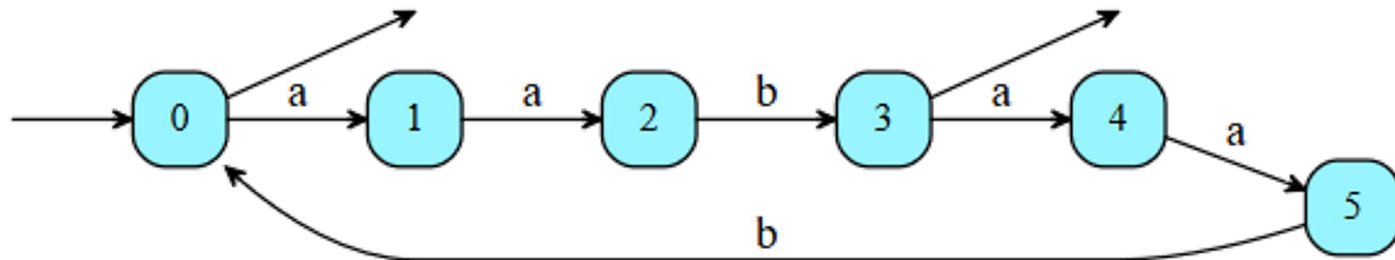
Minimization: example



This automaton is deterministic

states: {0, 1, 2, 3, 4, 5}

Minimization: example

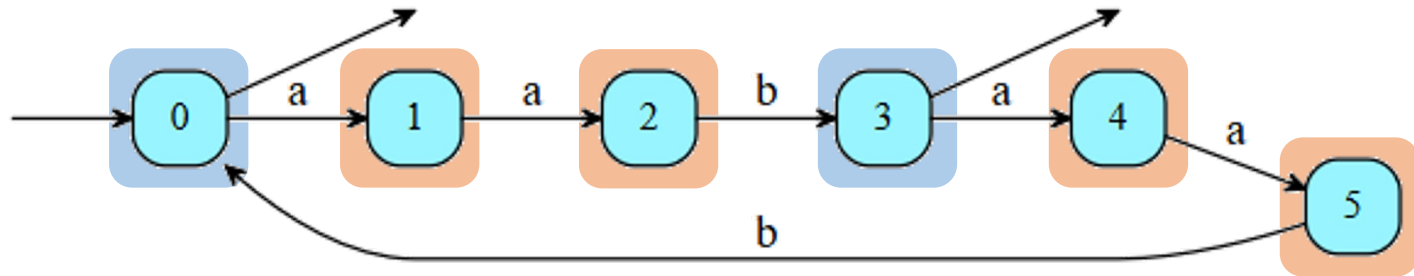


states : $\{0, 1, 2, 3, 4, 5\}$

we have $\{0, 3\}$ finals and $\{1, 2, 4, 5\}$ not finals

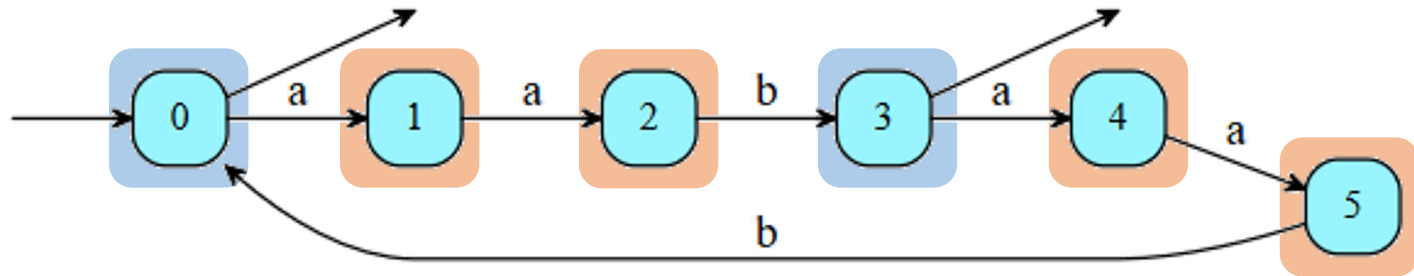
$\Rightarrow$ we need to split the initial partition in two.

Minimization: example



states: $\{0, 3\}$ $\{1, 2, 4, 5\}$ (2 partitions)

Minimization: example

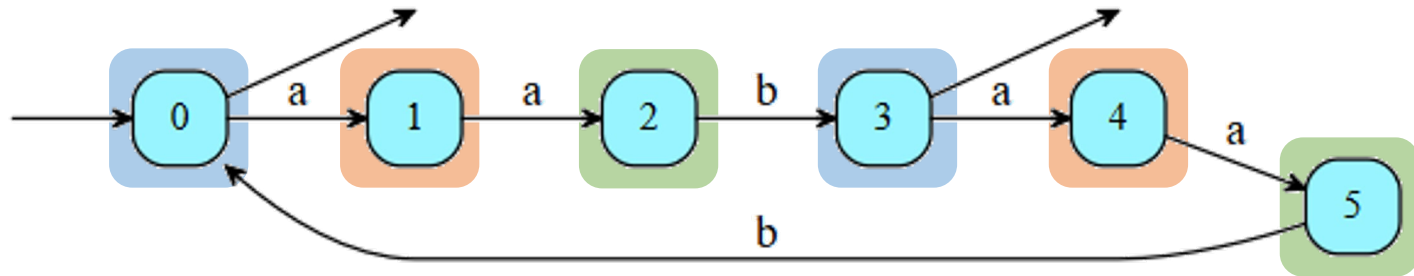


states: $\{0, 3\}$ $\{1, 2, 4, 5\}$

$0 \xrightarrow{a} \blacksquare$ and $1 \xrightarrow{a} \blacksquare$, same for $\xrightarrow{b}$: OK

$2 \xrightarrow{b} \blacksquare$ and $5 \xrightarrow{b} \blacksquare$, but 1, 4 block : need to split $\blacksquare$

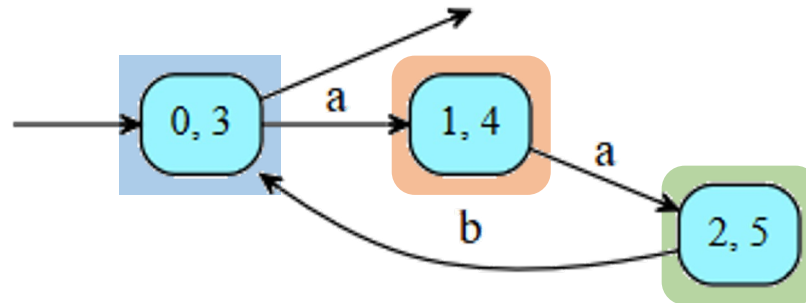
Minimization: example



states: $\{0, 3\}$ $\{1, 4\}$ $\{2, 5\}$ (3 partitions)

we have no more partitions to split

Minimization: example



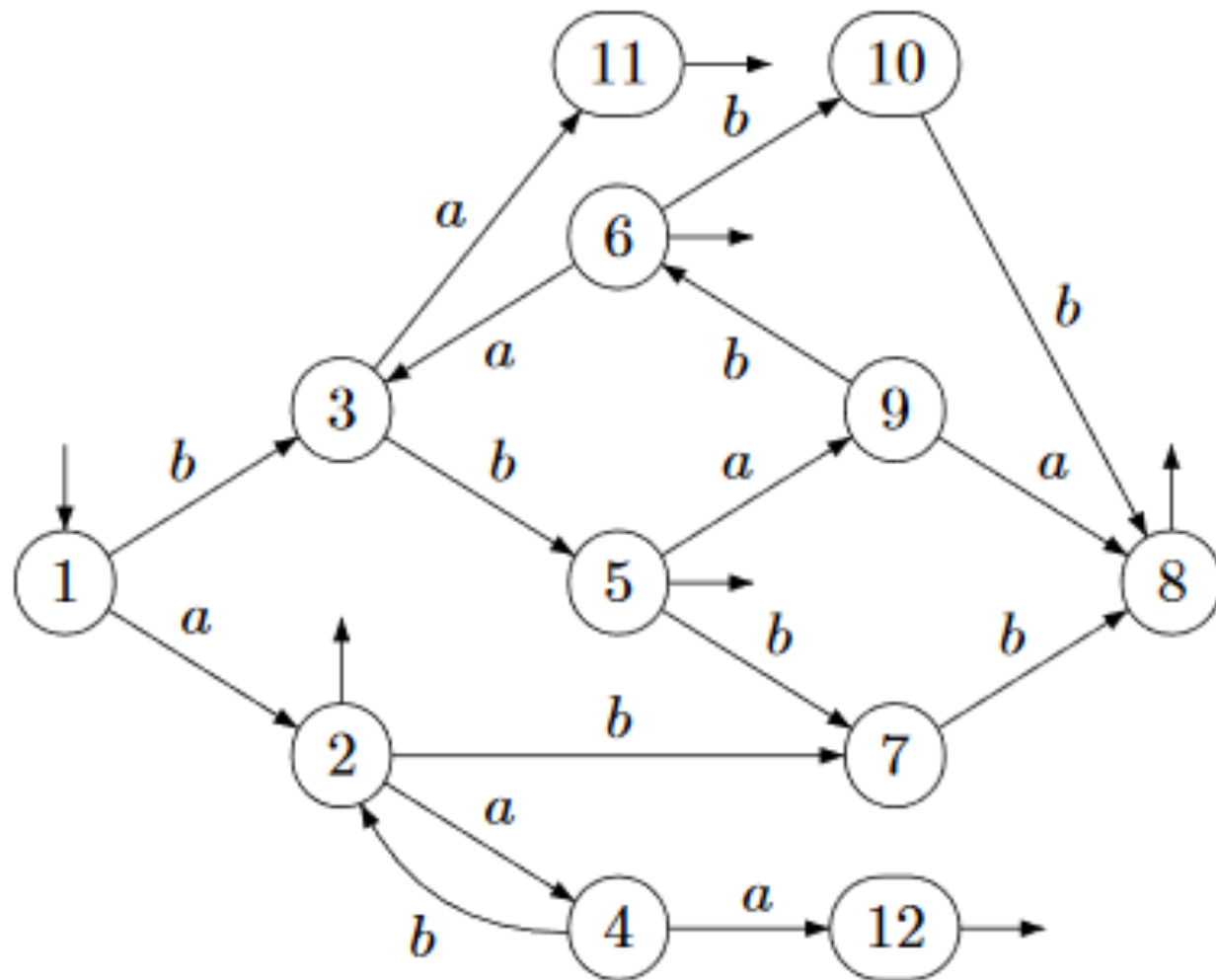
states: $\{0, 3\}$ $\{1, 4\}$ $\{2, 5\}$

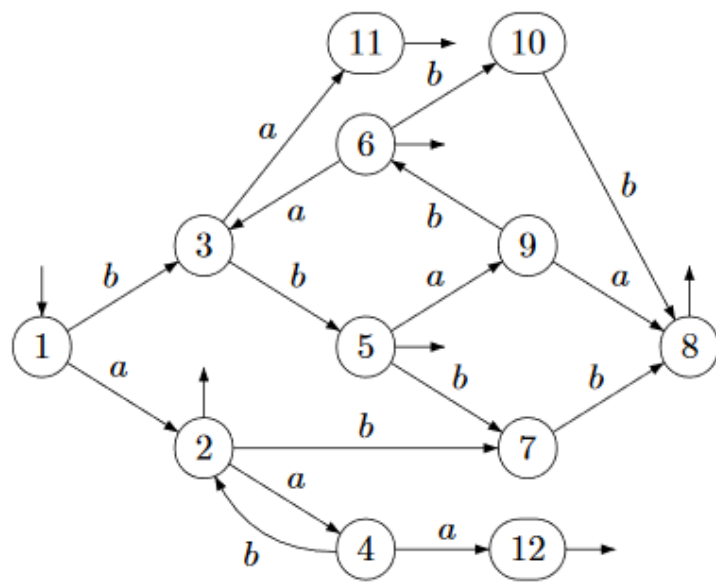
we obtain the minimal DFA by *fusing* states in the same partition together

Minimization: Hopcroft

- This algorithm is due to Hopcroft (1971)
 - requires a complete automaton
- We start from the *coarsest* relation $\{Q\}$ and stop when there are no more partition to split $\Rightarrow$ so we split at most n times ($n = |Q|$)
this uses a *coinductive proof principle*.
- complexity is $n |\Sigma| \log n$
- There is another $\sim$ algorithm due to Moore
- In each case, it requires to work on DFA

Example





Example: $\equiv$ between regexes

- $R_1 = (a + b)^*$
- $R_2 = (a^* + b^*)^*$
- $R_3 = (a^* + b.a^*)^*$
- $R_4 = (a^* + b)^*.a^*$

$$D(R(D(R(\mathcal{A}))))$$

Minimization: Brzozowski

- Start from a DFA $\mathcal{A}$ that is complete
- Reverse $\mathcal{A}$ (take its mirror image)
 - switch the direction of the “arcs”
 - the result may not be deterministic
- Determinize the result
 - the result is a minimal DFA for $\widehat{\mathcal{L}(\mathcal{A})}$
- Reverse the automata and determinize again

$\Rightarrow$ produces a minimal automata equivalent to $\mathcal{A}$

Equivalence between NFA

- Minimization is not a solution: too complex
- Finding a solution without determinization is complex, but some recent advances: Raskin (2006), Pous (2013)

Rational Languages

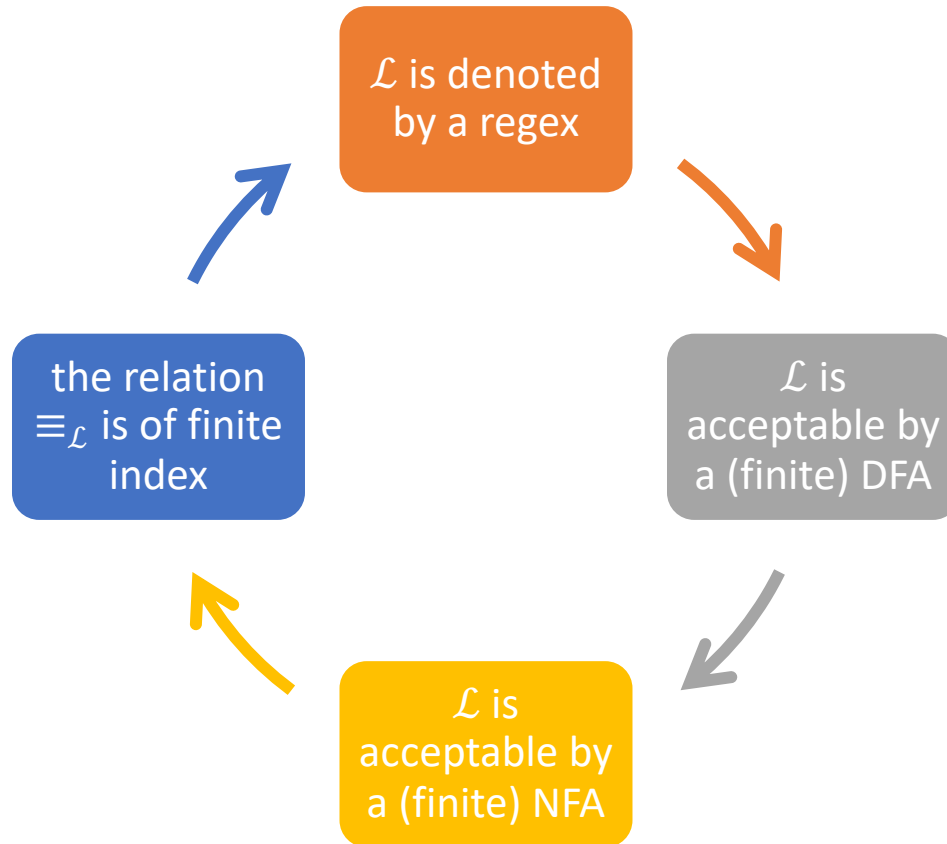
reminder

Equivalent characterization

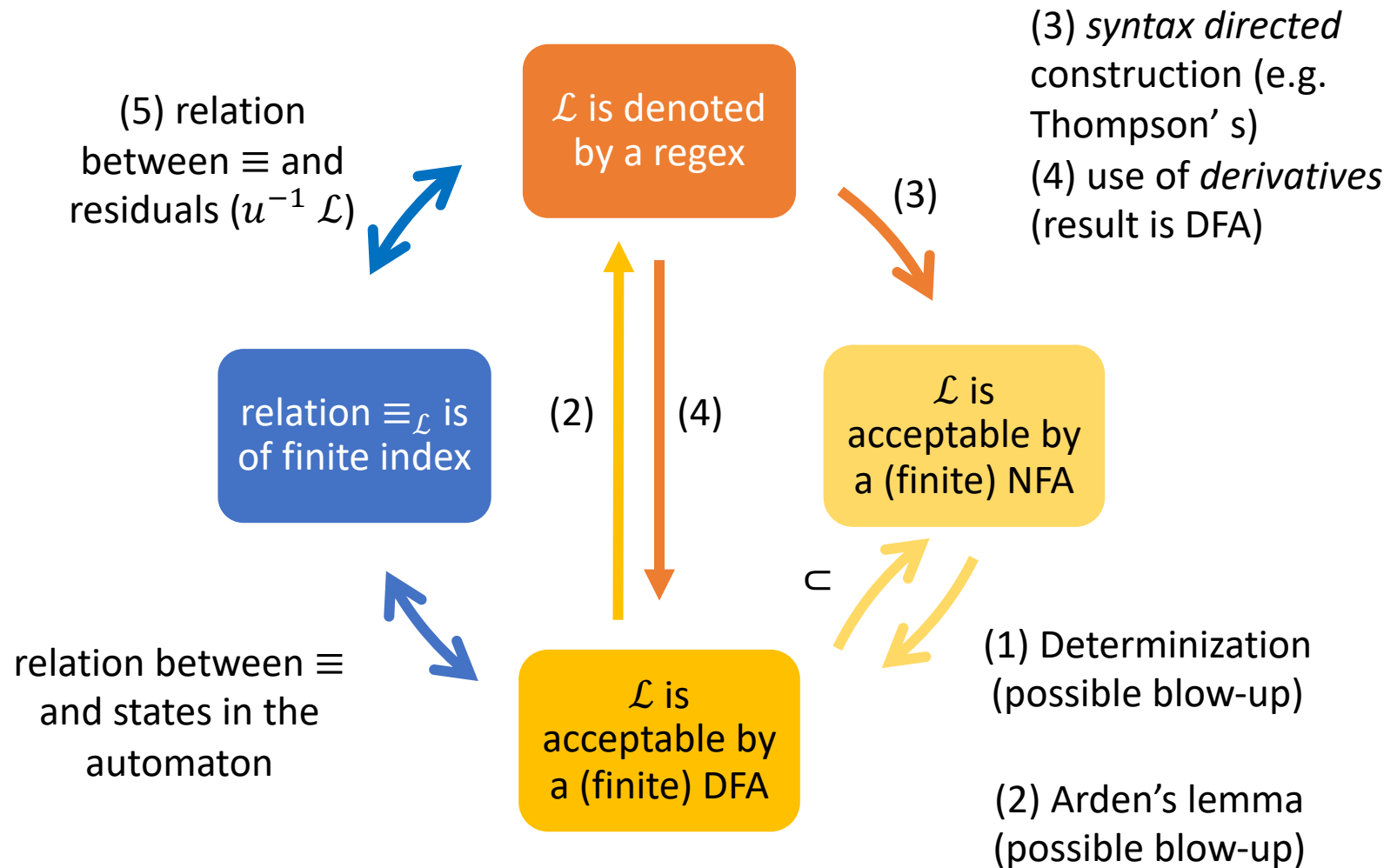
For a language $\mathcal{L}$, the following conditions are equivalent:

1. $\mathcal{L}$ is denoted by a regex
2. $\mathcal{L}$ is acceptable by a (finite) DFA
3. $\mathcal{L}$ is acceptable by a (finite) NFA
4. the relation $\equiv_{\mathcal{L}}$ is of finite index
5. $\mathcal{L}$ is generated by a right-linear grammar

Equivalent characterizations



Equivalent characterizations



Application of Myhill-Nerode th.

Are those languages rational ?

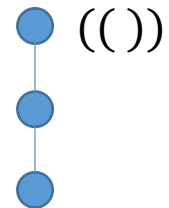
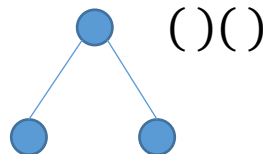
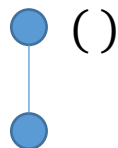
1. $\{ a^n b^n \mid n \geq 0 \}$
2. $\{ a^n b^n \mid n < 1\,000\,000 \}$
3. $\{ a^n b^m \mid n, m \geq 0 \}$
4. $\{ a^n b^m \mid n + m \equiv 0 [5] \}$
5. Dyck languages, containing (well-parenthesized) words of the form $\epsilon, a \bar{a}, a \bar{a} a \bar{a}, a a \bar{a} \bar{a}, \dots$
if $u, v \in \mathcal{L}$ then $a u \bar{a} \in \mathcal{L}$ et $u v \in \mathcal{L}$
imagine that $a = '('$ and $\bar{a} = ')'$

Dyck languages

Dyck languages $\equiv$ containing (well-parenthesized) words of the form ϵ , $a \bar{a}$, $a \bar{a} a \bar{a}$, $a a \bar{a} \bar{a}$, ...

Imagine that $a = '('$ and $\bar{a} = ')'$
words of the form ϵ , $()$, $()()$, $(())$, ...

Equivalently: $u, v \in \mathcal{L} \Rightarrow a u \bar{a} \in \mathcal{L}$ and $u v \in \mathcal{L}$



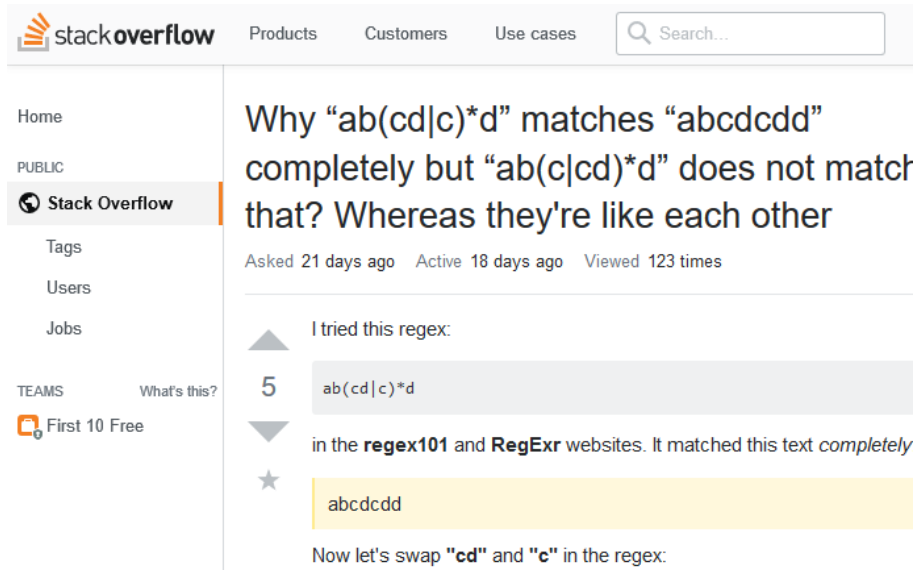
Automata

Conclusion

What you should have learned

- There are different views to the same problem:
operational, declarative, denotational
- We can better understand a problem when we have different ways to look at it
- C. S. problems can (sometimes) be answered using simple abstractions (discrete maths) and lead to interesting questions about complexity
⇒ what are the limits ?
- It has applications in practice, ...

... or not



The screenshot shows a Stack Overflow page with the following details:

- Header:** Stack Overflow logo, navigation links (Products, Customers, Use cases), and a search bar.
- Left Sidebar:** Home, PUBLIC, Stack Overflow (selected), Tags, Users, Jobs, TEAMS, What's this?, First 10 Free.
- Question Title:** Why "ab(cd|c)*d" matches "abcdcdd" completely but "ab(c|cd)*d" does not match that? Whereas they're like each other
- Metadata:** Asked 21 days ago, Active 18 days ago, Viewed 123 times
- Answer:** A single answer with a score of 5. The text of the answer is: "I tried this regex: ab(cd|c)*d in the **regex101** and **RegExr** websites. It matched this text *completely* abcdcdd". The text "abcdcdd" is highlighted in yellow.
- Follow-up:** Below the answer, it says "Now let's swap 'cd' and 'c' in the regex:".

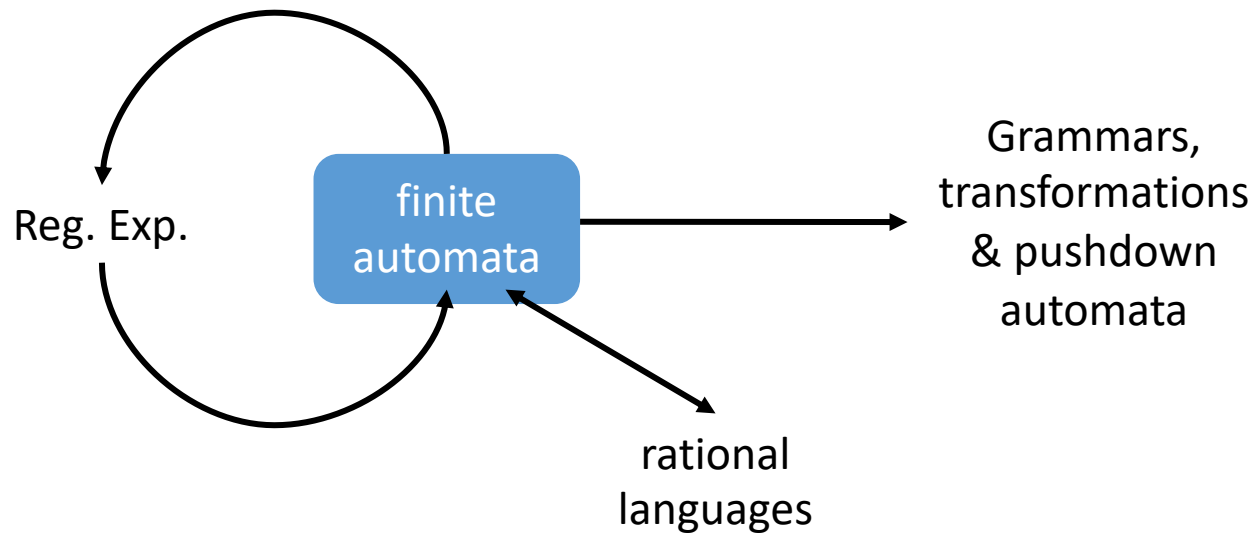
Why does $ab(cd + c)^*d$ matches *a b c d c d d* completely

But $ab(c + cd)^*d$ matches *a b c d c d d*

This is a problem/question about ambiguity

Automates et Langages

chapitre 4



Automata

quick refresher + an application to string searching

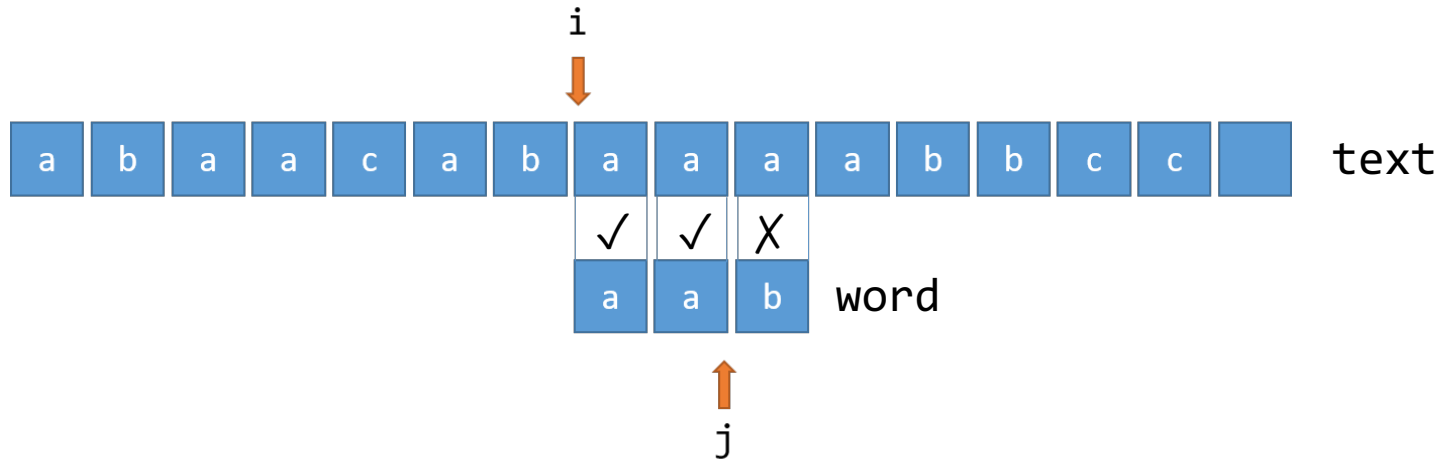
String searching

Problem: find the occurrences of a given word (the needle) inside a, usually much larger, text (the haystack)

- a simple problem (linear complexity)
- an important problem, with many applications: log analysis ; security ; DNA and protein sequences, string mining

An instance of a more general problem $\Rightarrow$ pattern matching

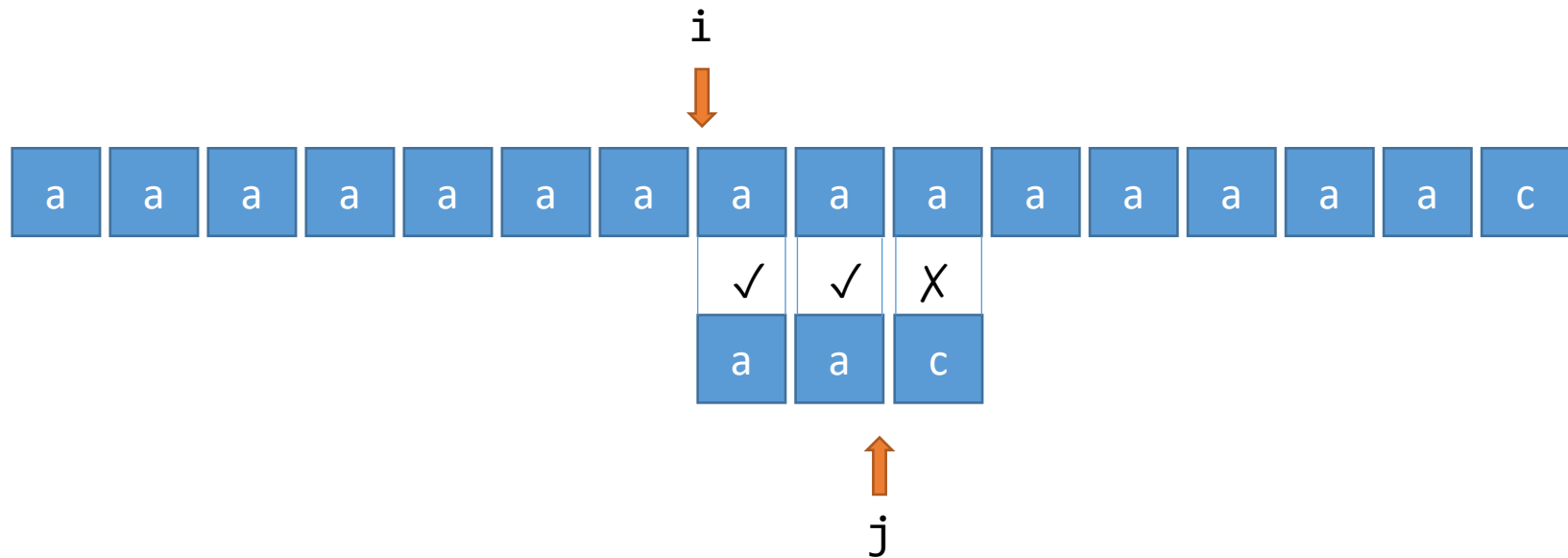
String searching: $O(n)$



```
func simple(word, text []byte) int {  
    for i = 0; i < len(text); i++ {  
        for j = 0; j < len(word); j++ {  
            if i+j == len(text) { return -1 }  
            if text[i+j] != pattern[j] { break }  
        }  
        return i }  
}
```

String searching: $k \times n$

Find a needle (size k) in a haystack (of size n)

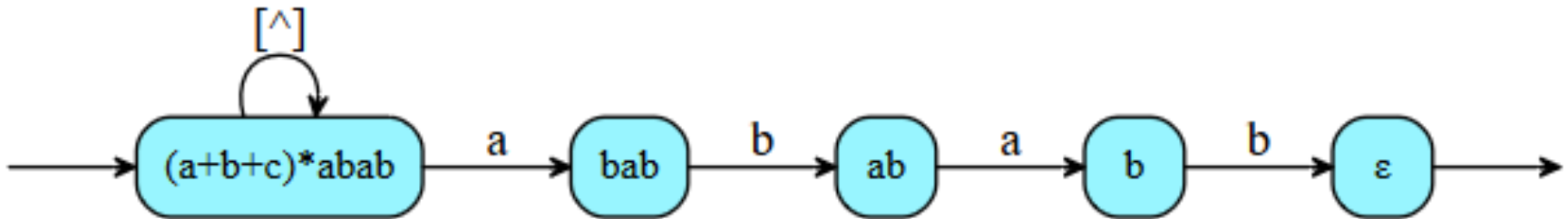


$\Sigma = \{ A, C, G, T \}$

We are in a case where the constants are important; two $O(n)$ algorithms are not equivalent. Can we do it in time n ?

Idea: use a DFA

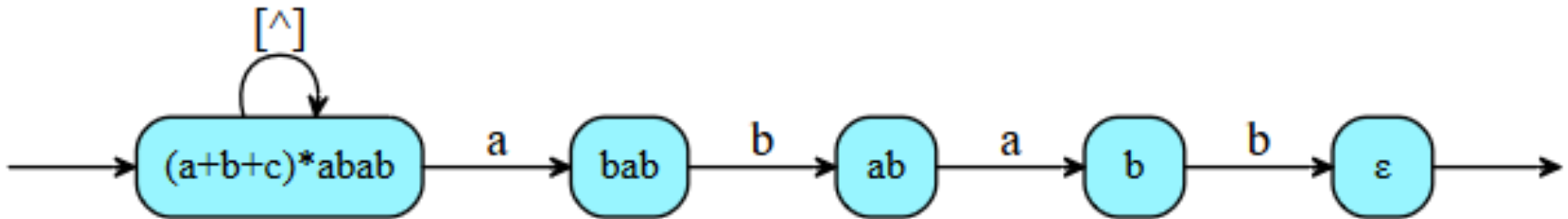
- Imagine we want to match the string $a b a b$
- We want to match every possible occurrences



here $[^]$ means Σ (and $[^a]$ is $\Sigma \setminus \{a\}$)

Idea: use a DFA

- Imagine we want to match the string $a b a b$
- We want to match every possible occurrences

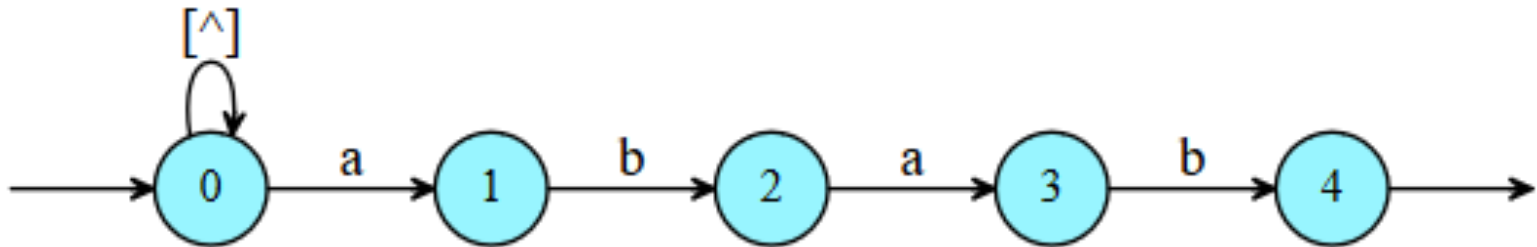


Drawbacks:

- are we sure to match all occurrences? Imagine the input $a b a b a b$ (that has 2 occurrences)
- non-deterministic $\Rightarrow$ we need to use “4 registers” (not optimal !?) or to use backtracking (not practical)

here $[^]$ means Σ (i.e. $\Sigma \setminus \emptyset$)

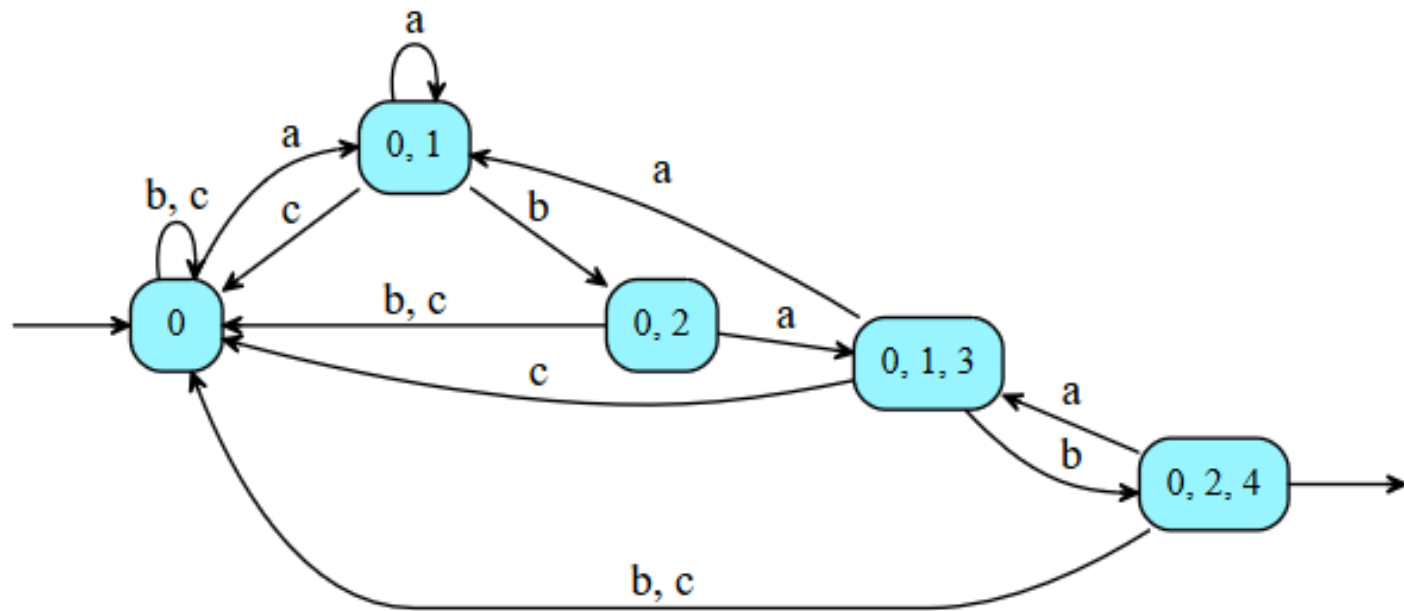
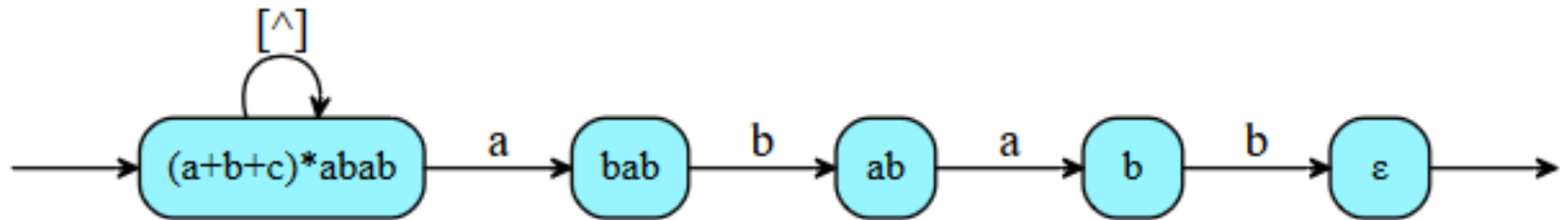
Déterminization



$\text{det}(\mathcal{A})$ est un tuple $(Q', \Sigma, \delta', q'_I, F')$ où :

- $Q' = 2^Q = \mathcal{P}(Q)$ (powerset)
- $\Sigma =$ même alphabet que $\mathcal{A}$
- $q'_I = \epsilon F(I)$
- $F' = \{ S \mid S \cap F \neq \emptyset \}$
- $\delta' \in (Q' \times \Sigma) \rightarrow Q'$: fonction de transition

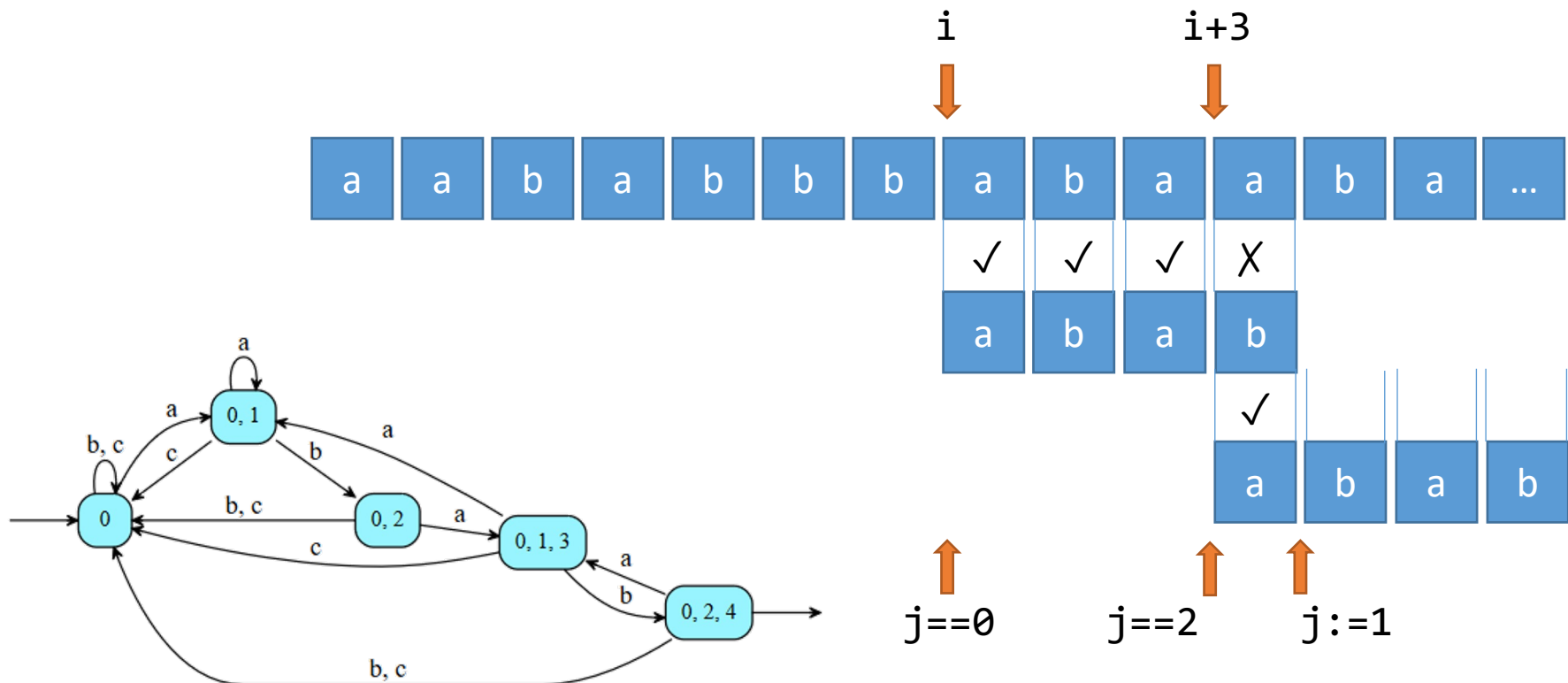
Idea: determinization



preprocessing time in $k \cdot |\Sigma|$

String matching: an observation

when we “fail” to match the next symbol, we do not always need to start over



String matching: an observation

- When matching fails at position j , it is enough to remember the longest prefix of word that is a suffix of word[0.. j]

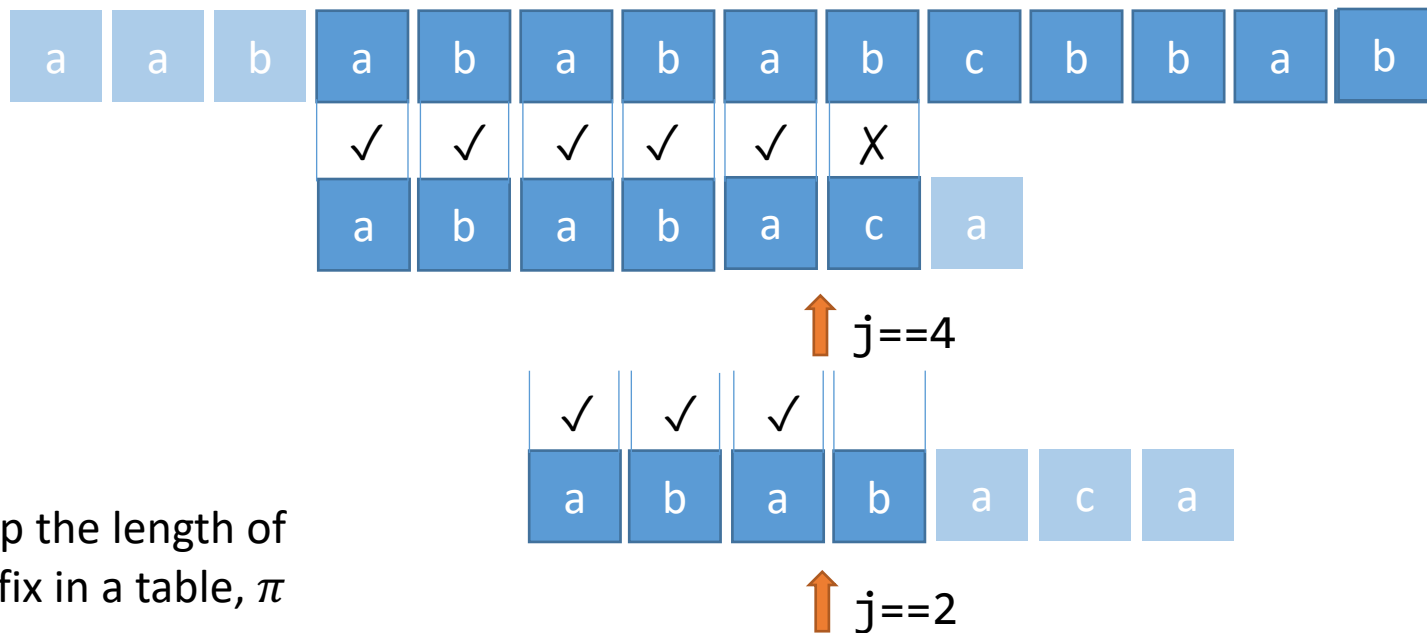
a b a b a b a b c a

a b a b a *b c a*

prefix suffix

String matching: an observation

- When matching fails at position j , it is enough to remember the longest prefix of word that is a suffix of word[0.. j]



We keep the length of the prefix in a table, π

String matching: an observation

- When matching fails at position j , it is enough to remember the longest prefix of word that is a suffix of word[0.. j]

word	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>
π	0	0	1	2	3	4	5	6	0	1

if we have already matched *a b a b a b* and fail at matching the next *a* (when $j=6$), the next “possibly useful” shift is $j=4$

String matching: an observation

- When matching fails at position j , it is enough to remember the longest prefix of word that is a suffix of word[0.. j]

word *a b a b a b a b c a*
 π 0 0 1 2 3 4 5 6 0 1

<i>a b a b a b d</i>	
<i>a b a b a</i>	$\pi(6) = 4$
<i>a b a</i>	$\pi(4) = 2$
<i>a</i>	$\pi(2) = 0$

String matching

- This approach $\equiv$ *Knuth-Morris-Pratt* algorithm (1970)
- Other algorithms use hashing $\Rightarrow$ Karp
 - good solution for approx./randomized pattern matching
- A state of the art algorithm is Boyer-Moore (1977)
 - scans the needle from right $\rightarrow$ left
 - part of the C++ STL, still used in grep, ...
- Aho–Corasick algorithm
 - search multiple strings: grep -F
 - “[...] constructs a finite-state machine that resembles a trie with additional links between internal nodes.”

Another cool construction

- Levenshtein automaton
- fuzzy string matching (wrt the *edit distance*)

$u =$ a b a a c c
 $v =$ a b a d c

$$\|u - v\| = 2$$

Languages and complexity

Some complexity theory

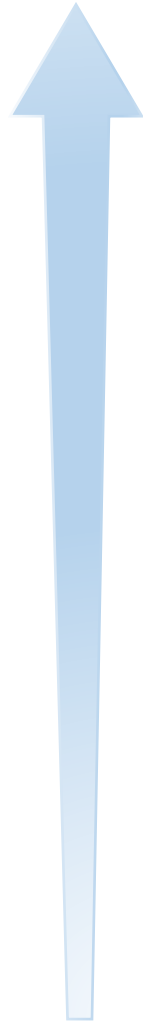
Languages

- (Formal) Languages are subsets of Σ^*
 - *this is a very general definition*
 - we have concentrated on *regular languages*, but not all languages are regular
- Regular/Rational languages $\equiv$ a class of languages with good properties
 - multiple (unrelated!) ways to define regularity
 - closed by various operations: $+$, $\times$, $\div$, $\cap$, $\star$, $\overline{}$, $\sim$, ...
 - many **decidable properties**
 - a “meaningful” **complexity class**, REG

Languages: questions

- What are some examples of **properties** and **problems** ?
- What do you mean by **complexity class** of a set of languages
- Can we define classes that are **simpler**, or **more complex** than REG ?

Languages: hierarchy

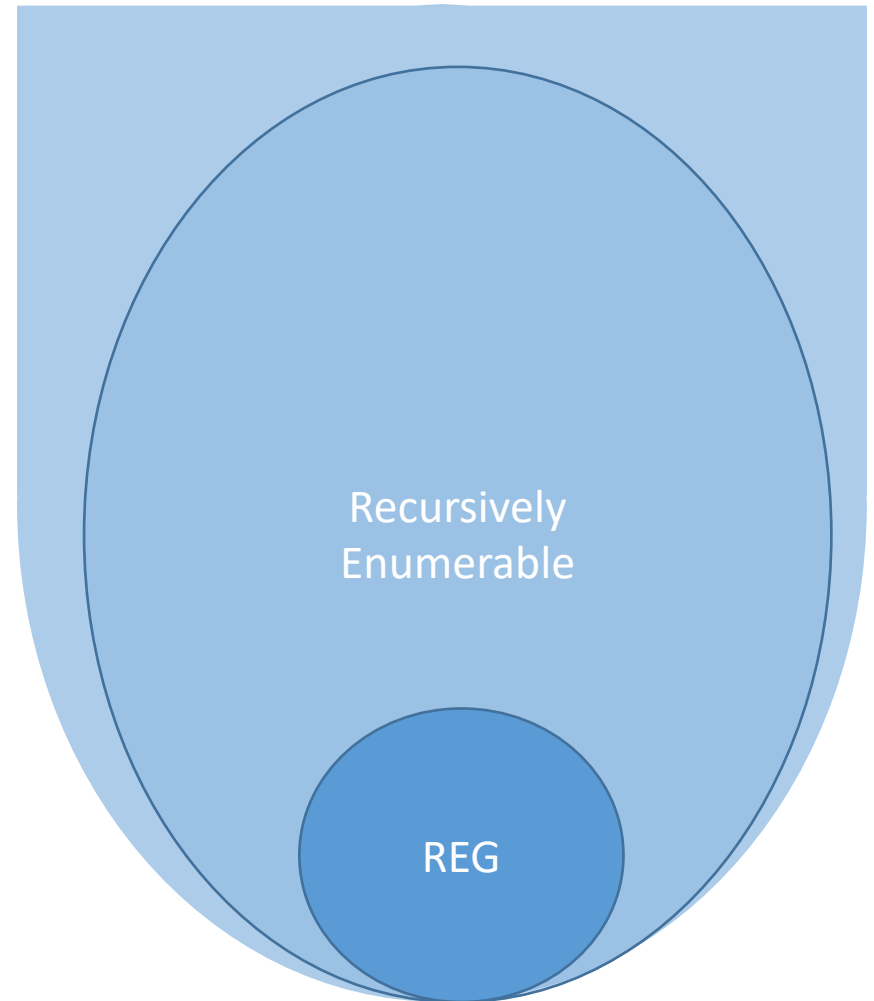


$\mathcal{P}(\Sigma^*)$

in between ?

REG

even simpler ?



Languages: problems

- We have often tried to prove whether two regex / FSA “are the same”
- A more basic question if *containment* : $\mathcal{L}_A \subseteq^? \mathcal{L}_B$?

Languages: problems

There are simpler questions:

membership: is a given word u in $\mathcal{L}_A$?

emptiness: is it true that $\mathcal{L}_A = \emptyset$?

does $\mathcal{L}_A$ contains ∞^{ly} many words ?

universality: does $\mathcal{L}_A = \Sigma^*$?

- **Good news:** almost all problems of interest are decidable in REG

REG: complexity

Membership problem: $u \in \mathcal{L}_A$?

```
func member(u []byte) bool {  
    st := q0  
    for i = 0; i < len(u); i++ {  
        st = delta(st, u[i])  
    }  
    return isfinal(st)  
}
```

We can test membership using a “machine” that has one register (storing the current state) and a table for storing δ and F

REG: complexity of membership

Assume $\mathcal{A}$ is a DFA with n states.

We can test membership using a “machine” that has **one register** (storing the current state) and a table for **storing δ and F**

$\log n$ bits of
writable data

non-writable
data

Hence problem is in DLOGSPACE (also called L)

REG: complexity of membership

Membership for DFA is in $DLOGSPACE-c$

Membership for NFA is in $NLOGSPACE$ ($L \stackrel{?}{=} NL$)

$\equiv$ membership for regex

Same complexity than deciding whether there exists a path between given vertices in a directed graph

Considered feasible

Complexity

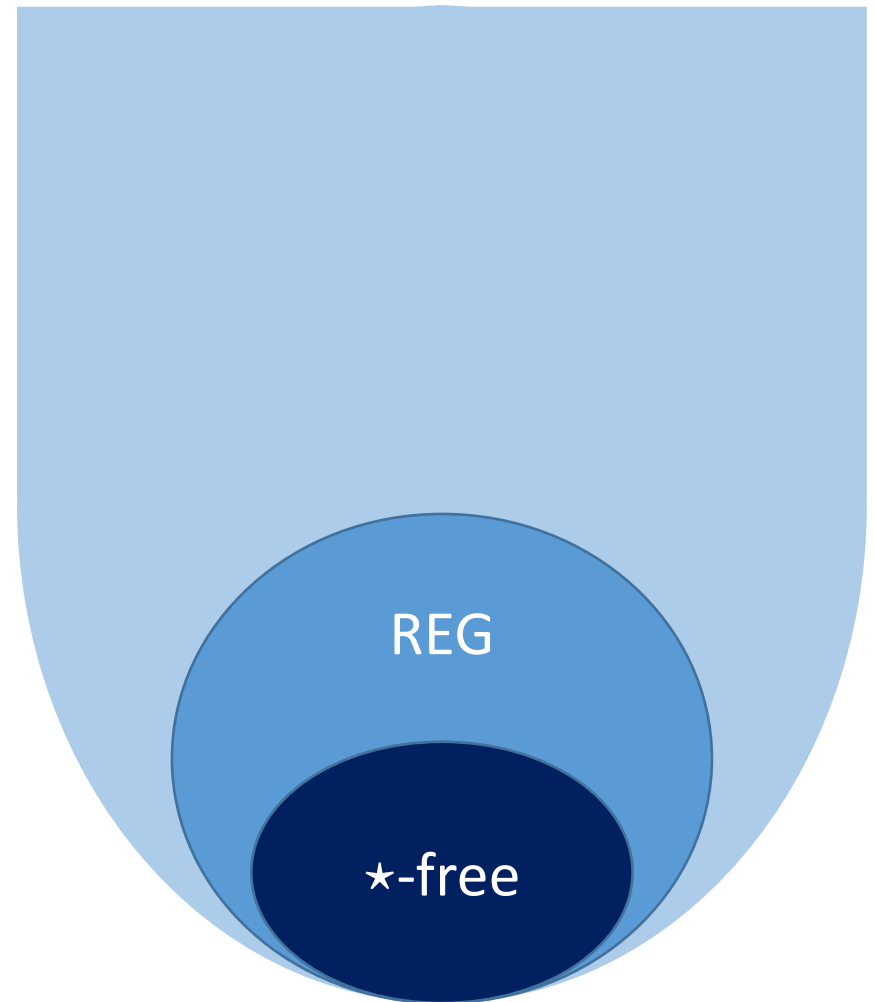
- Emptiness: $\mathcal{L} \stackrel{?}{=} \emptyset$
is in NLOGSPACE-c for both NFA and DFA
- Equivalence: $\mathcal{L}_1 \stackrel{?}{=} \mathcal{L}_2$
is PSPACE-c for DFA, NFA and regex

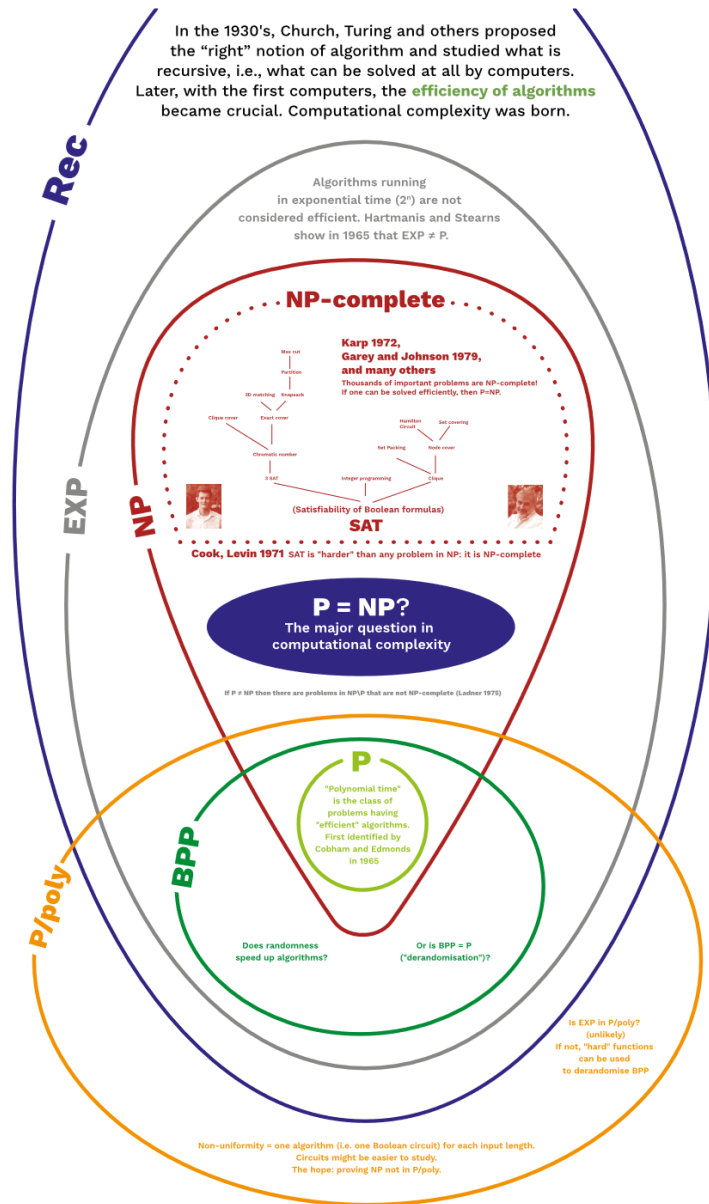
Suspected infeasible

Languages hierarchy: star-free

star free languages $\equiv$
corresponds to
generalized regex ($\bar{e}$)
without $\star$

- example: $\overline{\mathbf{0} a a \mathbf{0}}$
- count.-ex.: $(aa)^\star$



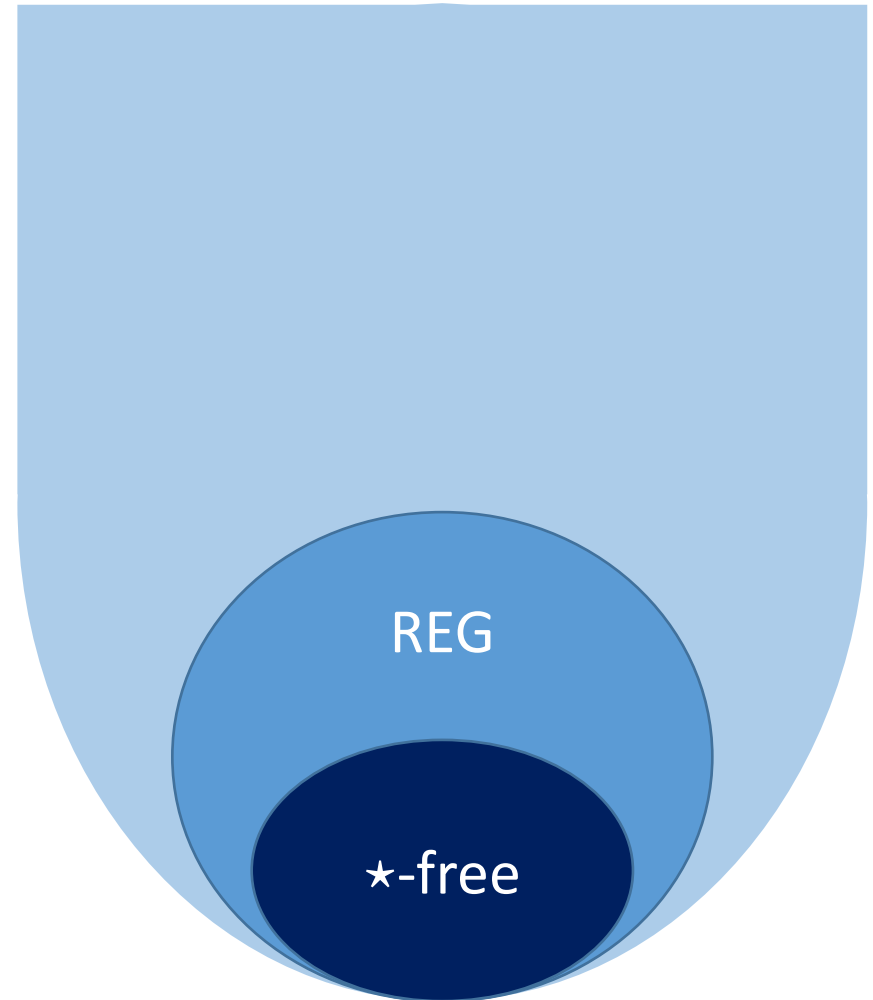


Rec
problems that can
be solved by computers

EXP
Exponential
time

NP
Nondeterministic Polynomial time:
solutions can be **verified** efficiently

BPP
Bounded-error Probabilistic
Polynomial time



Modelling with automata

Another example of modelling

Swimming Pool

Model the operations of a swimming pool

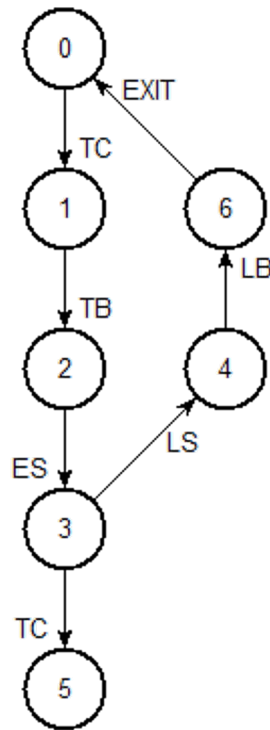
- A swimming pool comprises c cabins to change and p baskets to deposit clothes.
- A user can enter the pool only if a cabin is free.
- Once he has a cabin, he has to wait for a basket to change and deposit his clothes.
- Then it releases the cabin and enter the swimming pool.
- He can leave only if a cabin is free.
- After changing he frees the cabin and basket.
- Finally he leaves the pool.

Exercise: only one basket

- A user can **enter the pool** only if a cabin is free. TC: Take Cabin
- Once he has a cabin, he has to wait for a basket to **change and deposit** his clothes. TB: Take Basket
- Then it releases the cabin and **enter the swimming pool**. ES: Enter Basin
- He can **leave** only if a cabin is free. LS: Leave Basin
- After changing he **frees the cabin and basket**. LB: Leave Basket
- Finally he **leaves the pool**. EXIT: exit pool

Exercise: one basket

The swimming pool has $c = 1$ cabins to change and $p = 1$ baskets to deposit clothes.



1 basket,
1 cabin

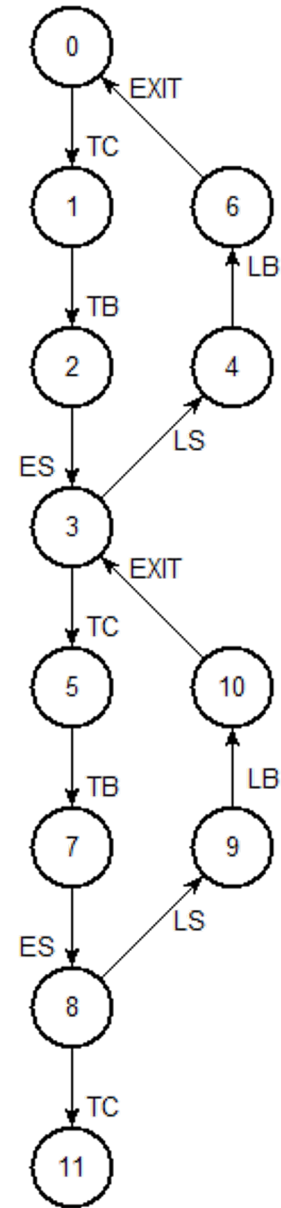
Exercise: two baskets

The swimming pool has $c = 1$ cabins to change and $p = 2$ baskets to deposit clothes.

Exercise: two baskets

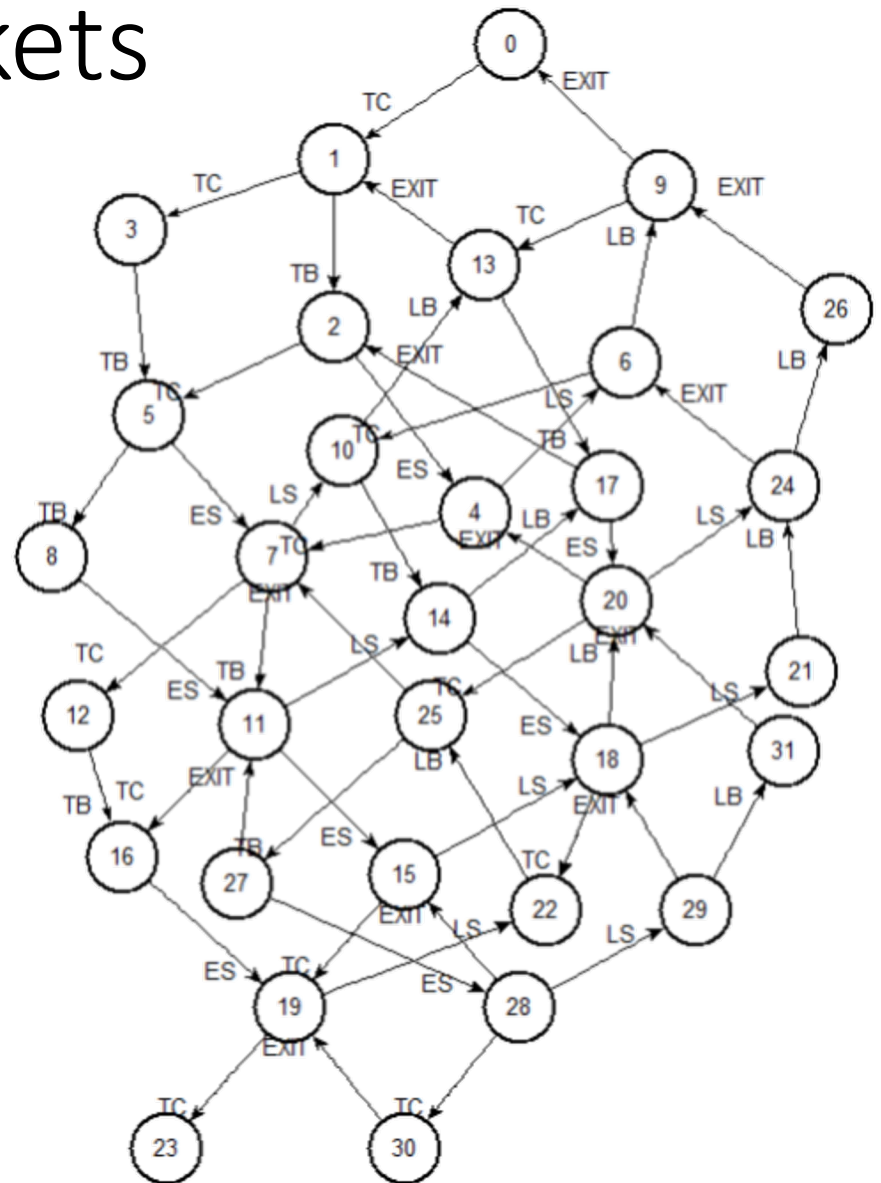
The swimming pool has $c = 1$ cabins to change and $p = 2$ baskets to deposit clothes.

2 baskets,
1 cabin



Exercise: two baskets

Would you model the system with 5 cabins and 8 baskets ?



2 baskets,
2 cabins